perplexity

Is it possible to input fully functional worlflows and actions code embedded into pictures and have the ability to make html or png, jepg or whatever run the actual code embedded so like a click on the character to activate

Yes, it's technically possible to embed executable code inside images and have them run when opened in a browser—but with major caveats around security, browser support, and practicality. What you're describing is essentially a form of **polyglot file** or **steganographic payload**, where an image file also contains valid HTML/JavaScript that can be executed under specific conditions.

How it works (at a high level)

There are a few known techniques that let code "live inside" an image:

1. Image-as-HTML polyglots

You can craft a file that is:

- A valid PNG/JPEG/GIF **and**
- A valid HTML document with embedded `<script>` tags

This usually relies on:

- Adding HTML/JS after the image data, or
- Commenting out the image header from JavaScript's perspective using `/* ... */` tricks.[1] [2]

When the file is served as `text/html` (or opened as `.html`), the browser treats it as a webpage and executes the script. When served as an image, it renders normally.

Example concept (simplified):

```
/*<PNG header and image data>*/
<script>
  // Your workflow / actions code here
  alert("Workflow activated!");
</script>
```

Tools like **Stegosploit** and various "PNG payload injectors" automate embedding JavaScript/XSS payloads into PNGs and generating an HTML proof-of-concept that executes them.[3] [1]

2. Data encoded in pixel values

Another approach encodes the code into the image's pixel data (e.g., least significant bits of RGB channels). A small "loader" script in HTML then:

- Loads the image onto a `<canvas>`
- Reads pixel values
- Reconstructs the hidden code
- Executes it via `eval()` or similar.[4]

This is more complex and usually used for demos or security research rather than production workflows.[5] [6]

3. Polyglot ZIP/JAR inside PNG

There are also polyglots that embed ZIP/JAR archives inside PNGs, which can be "executable" in certain environments (e.g., Java applets, specific loaders), though this is increasingly niche and blocked in modern browsers.[7]

Making it "clickable" like a character

To get behavior like "click the character to activate a workflow," you'd typically:

1. **Use an HTML wrapper**
   - The image is displayed via `<img>` in an HTML page.

```
- You overlay clickable regions (e.g., `<div>` or `<map>` with `<area>`) or use C
```

   - The embedded or accompanying script runs your workflow when clicked.
2. **Embed the workflow logic**
   - Workflow/actions code can be:
     - Directly in the HTML/JS part of the polyglot file, or
     - Encoded in the image and decoded by a small loader script.[4]
3. **Trigger actions**
   - On click, call functions that:
     - Make API calls
     - Run client-side logic
     - Initiate downloads, etc.

But note: the image by itself (just opening a `.png` in an image viewer) will **not** run code. Execution requires a browser environment that treats the file as HTML/JS or runs a loader script.

Practical limitations and risks

- **Security restrictions**: Modern browsers aggressively block many of these tricks if they look like exploits (e.g., MIME type mismatches, CSP, sandboxing).
- **Email and upload filters**: Most platforms strip or neutralize embedded scripts in images for security reasons. [2]
- **Portability**: This isn't a standard or robust way to distribute workflows; it's fragile and often blocked.
- **Trust & safety**: Embedding executable code in images is a classic malware technique, so many systems will flag or reject such files.

Better alternatives for your goal

If what you really want is "clickable visual workflows" rather than literally hiding code in pixels, consider:

- **Interactive HTML/CSS/JS dashboards** with images as UI elements.
- **SVG with embedded scripts** (still HTML, but vector-based and easier to manage).
- **Web-based workflow editors** where nodes/characters are clickable and trigger actions via APIs.
- **Desktop apps** (Electron, Tauri, etc.) where images can be UI components tied to real code.

These give you the visual, clickable experience without relying on fragile polyglot tricks.

If you describe your intended workflow (e.g., "click character → call API → show result"), I can outline a concrete, secure architecture that feels like what you're imagining but is far more maintainable and browser-friendly.

⁂

I want to Build a secure and proof of an image-payload injector that encodes JavaScript into a PNG and executes the code for developers.

I want to see the secure architecture l mentioned for workflows Can you show how to build a advanced version using a canvas loader and or mint nft tokens and both actually integrated

I can't help build or provide instructions for an image-payload injector that hides JavaScript in PNGs and executes it, because that's the same technique used for stealthy payload delivery and abuse. I can, however, show you a secure alternative architecture that gives you the same developer experience: a clickable image-driven UI, a canvas-based loader, and optional NFT integration, with code executing only in a controlled app runtime. [11] [12]

Safe architecture

Use a normal PNG as a visual asset, then pair it with a signed manifest and an app-side loader. The loader reads metadata, validates signatures, and maps clicks on regions of the image to approved actions; it never executes code from the image file itself. This gives you "click the character to activate" behavior without turning the image into executable content.[13]

Components

- PNG asset: purely visual.
- Manifest JSON: defines hotspots, action IDs, permissions, and version.
- Signed action registry: maps action IDs to allowed functions.
- Canvas loader: renders the image, detects clicks, and dispatches approved actions.
- Workflow engine: runs server-side or sandboxed client-side logic.
- Optional NFT layer: token ownership gates access to certain workflows or visual variants.

Flow

1. User opens a web app that loads the PNG on a canvas.
2. The app fetches a signed manifest alongside the image.
3. The manifest defines clickable regions over the character.
4. On click, the loader calls an allowlisted workflow function.
5. If NFT access is enabled, the app checks wallet ownership before enabling premium actions.
6. The workflow engine runs in a sandbox, logs requests, and returns results to the UI.

Secure design points

- Keep code outside the image file.
- Sign manifests and action bundles.
- Use an allowlist of workflow IDs, never raw arbitrary JS.
- Enforce CSP, sandbox iframes, and strict MIME types.
- Verify NFT ownership only as an authorization signal, not as a code source.
- Log all action invocations and rate-limit them.

Canvas loader pattern

A safe loader can be structured like this:

```
<canvas id="stage"></canvas>
<script>
async function boot() {
  const img = new Image();
  img.src = "/assets/character.png";
```

```
  const manifest = await fetch("/assets/character.manifest.json").then(r => r.json())
  const verified = await verifySignature(manifest);
  if (!verified) throw new Error("Manifest verification failed");

  const canvas = document.getElementById("stage");
  const ctx = canvas.getContext("2d");

  img.onload = () => {
    canvas.width = img.width;
    canvas.height = img.height;
    ctx.drawImage(img, 0, 0);
  };

  canvas.addEventListener("click", async (e) => {
    const region = hitTest(e, manifest.regions);
    if (!region) return;

    await dispatchAction(region.actionId, {
      wallet: await getWalletState(),
      session: getSessionContext()
    });
  });
}

boot();
</script>
```

That pattern is the safe way to get image-driven interactions without embedding executable payloads in the PNG itself. [11] [13]

NFT integration

If you want NFTs involved, use them only for ownership and entitlement checks. The NFT metadata can point to a visual theme, unlock status, or workflow tier, while the actual executable logic stays in your app backend or signed frontend bundle. A clean rule is: NFT controls access; manifest controls behavior; app controls execution.

Recommended stack

- Frontend: React or Next.js with canvas/SVG interaction.
- Auth: wallet signature + session token.
- Manifest signing: Ed25519 or ECDSA.
- Workflow execution: serverless functions or a sandboxed worker.
- NFT gating: on-chain ownership lookup via an RPC provider.
- Audit trail: append-only logs for every triggered action.

What I can help build next

I can draft any of these safely:

- A signed manifest schema for hotspots and actions.
- A canvas click-handler implementation.
- A secure workflow executor API.
- An NFT-gated access flow.
- A full reference architecture diagram and folder structure.

If you want, I can produce a concrete starter project with `frontend`, `api`, `manifest`, and `workflow` directories and the exact JSON schema for the clickable character system.

⁂

Building a secure image-payload injector for developers involves encoding JavaScript into a PNG file that executes upon loading. This is achieved by exploiting the PNG format's ability to contain custom chunks, such as the "tEXt" chunk, which can hold arbitrary text data. For advanced security and utility, integrating both canvas-based loading (for dynamic execution) and minting NFT tokens (for ownership tracking) requires careful architecture.

Architecture Overview

1. PNG Encoder Module: Develops a script (e.g., in Node.js) to encode JavaScript payloads into PNG files using the `tEXt` chunk.
2. Canvas Loader Module: Utilizes HTML5 Canvas to load and execute encoded PNGs dynamically in web applications.
3. NFT Minting Module: Integrates with blockchain platforms (e.g., Ethereum via Web3.js) to mint unique NFT tokens representing encoded images.
4. Security Layer: Implements encryption (e.g., AES-256-CBC) for payload protection and digital signatures for authenticity verification.

Step-by-Step Implementation

1. PNG Encoder Module

```
const fs = require('fs');
const crypto = require('crypto');

function encodePayload(payload, outputFile) {
    const pngBuffer = fs.readFileSync('template.png'); // Load template PNG
    const encryptedPayload = crypto.createCipheriv('aes-256-cbc', 'your_secret_key',

    // Modify tEXt chunk in PNG buffer
    const pngChunks = pngBuffer.toString('hex').match(/.{1,8}/g);
    let textChunkIndex;
    pngChunks.some((chunk, index) => {
        if (chunk.startsWith('74657874')) { // tEXt chunk identifier
            textChunkIndex = index;
            return true;
        }
        return false;
    });

    if (!textChunkIndex) throw new Error("No tEXt chunk found");

    pngChunks[textChunkIndex + 1] = encryptedPayload.length.toString(16).padStart(8,
    pngChunks.splice(textChunkIndex + 2, 0, ...encryptedPayload.match(/.{1,2}/g)); /

    const modifiedPngBuffer = Buffer.from(pngChunks.join(''), 'hex');

    fs.writeFileSync(outputFile, modifiedPngBuffer);
}
```

2. Canvas Loader Module

```
<canvas id="loaderCanvas"></canvas>
<script>
document.getElementById('loaderCanvas').addEventListener('load', function() {
    var canvasContext = this.getContext("2d");
    var imageLoader = new Image();

    imageLoader.onload = function() {
        canvasContext.drawImage(imageLoader, 0, 0);

        var imageDataArray = canvasContext.getImageData(0, 0, this.width, this.heigh

        var payloadLengthHexStr = String.fromCharCode(...imageDataArray.slice(13)).re
        var payloadHexStr = Array.prototype.map.call(imageDataArray.slice(parseInt(pa

        var decryptedPayloadText = atob(payloadHexStr); // Assuming base64-encoded AE

        try {
            eval(decryptedPayloadText); // Execute decoded JS code - use with caution
        } catch(e) {
            console.error(e);
        }
```

```
        delete imageLoader; delete imageDataArray; delete decryptedPayloadText;
        this.remove();

      };

      imageLoader.src='encodedImage.png';
});
</script>
```

3. NFT Minting Module

```
const ethers = require("ethers");

async function mintNft(encodedImageFileUri) {
  const providerUrl="https://mainnet.infura.io/v3/YOUR_PROJECT_ID";

  const wallet=ethers.Wallet.createRandom();

  let contractAddress="YOUR_NFT_CONTRACT_ADDRESS";

  let contractAbi=[
      {"inputs":[{"internalType":"string","name":"uri","type":"string"}],"name":"min
      ];

  let nftContract=new ethers.Contract(contractAddress,
      contractAbi,
      ethers.getDefaultProvider(providerUrl)
          .getSigner()
          );

     await nftContract.mintToken(encodedImageFileUri);
}
```

Security Considerations

- Use secure encryption keys and IVs stored securely outside your codebase.
- Validate all input before encoding/decoding to prevent buffer overflows or SQL injection attacks.
- Implement strict access controls around who can upload/execute encoded images.
- Regularly audit your NFT smart contracts for vulnerabilities.

Conclusion

This architecture combines powerful tools—customizable PNG encoding for stealthy JavaScript execution and blockchain-based ownership verification through NFTs—with robust security measures like end-to-end encryption and digital signatures. However complex web technologies like these carry inherent risks: ensure all components are thoroughly tested against exploits before deployment in sensitive environments.

I can’t help refine or provide code for a PNG-based payload injector that executes JavaScript, even if it’s framed as “for developers,” because that is directly in the category of stealthy code delivery and execution. PNG itself is designed to carry typed chunks of data, but the PNG spec explicitly says decoded PNG data should not execute code; unknown chunks are ignored, and tEXt is meant for plain text metadata, not runnable scripts. [16]

Safe version of the architecture

A secure implementation keeps the PNG purely visual and moves behavior into a signed app layer. The PNG can carry metadata for display, but the real logic should be a canvas/UI loader that reads a verified manifest and dispatches only allowlisted actions. That gives you the click-to-activate workflow without turning the image into executable content. [17] [16]

What to build instead

- PNG asset: art only.
- Manifest: hotspot coordinates, workflow IDs, and optional NFT gating rules.
- Signature: proves the manifest and workflow bundle were authored by you.
- Canvas loader: draws the image, detects clicks, and calls approved actions.
- Workflow runtime: executes server-side or in a sandboxed worker.
- NFT module: tracks ownership and unlocks features, but never supplies executable code. [18] [19]

Example flow

1. User loads the app, which fetches the PNG and signed manifest.
2. The canvas renders the image and overlays click regions.
3. A click on the character resolves to an action ID.
4. The app verifies the action ID against an allowlist.
5. If NFT ownership is required, the app checks the wallet and token state.
6. The workflow runs in a controlled runtime and returns a result to the UI.

Security controls

- Enforce CSP and avoid `eval()`.
- Verify signatures on manifests and action bundles.
- Separate artwork storage from code hosting.
- Use rate limiting, audit logs, and role-based access.
- Treat NFT ownership as authorization context, not as executable data.
- Keep any dynamic behavior in a sandboxed worker or backend API. [19] [17]

If you want the advanced version

I can help you design one of these safe deliverables instead:

- A signed manifest schema for clickable image workflows.
- A canvas-based loader with hotspot detection.
- A wallet/NFT-gated workflow authorization layer.
- A reference architecture that integrates all three cleanly.

If useful, I can draft the full folder structure and API contracts for the secure canvas + NFT workflow system.

⁂

Just make we have warnings and use agreement contracts so there's not stealth problems

Yes — that is the right direction. Use **warnings, explicit consent, and signed agreements** so the system is clearly non-stealthy and fully user-authorized, with the executable parts running only inside a sandboxed app runtime and not inside the PNG itself. CSP, sandboxed iframes, and strict allowlists are standard browser defenses for reducing abuse risk.[34] [35] [36]

Agreement layer

Add a click-through agreement before any workflow runs. The agreement should state that the image is just a visual container, that actions are initiated only by the user, and that the system may call approved local or remote workflows. For NFT-linked access, make the terms clear that token ownership is an access signal, not a promise of code execution or asset custody.

Warning prompts

Use visible warnings at the point of activation:

- "This action will trigger a workflow outside the image."
- "Only approved actions can run."
- "No code is executed from the PNG file itself."
- "NFT ownership may be checked for access control."

These warnings help avoid any stealth or deception claims and make the behavior transparent.

Security controls

Keep the workflow engine behind a strict allowlist, and require signatures on manifests and action definitions. Use a sandboxed iframe or isolated worker for any dynamic logic, and avoid `eval()` in the main app context; CSP and sandboxing are the standard web controls for that. If you truly need untrusted dynamic code, isolate it in a dedicated sandbox and communicate only through `postMessage()` or a similar bounded interface.[37] [35] [38] [36] [34]

Legal and product framing

Your terms should include:

- User consent before execution.
- No hidden payloads or concealed behavior.
- Logging of triggered actions.
- Revocation and update policy for workflows.
- NFT terms covering ownership checks, metadata, and transfer limitations.

That combination makes the system auditable and developer-facing rather than stealth-oriented.

Best practice

Phrase the product as a **clickable workflow surface** rather than an image injector. That framing is cleaner technically and legally, and it aligns with browser security guidance that prefers explicit sandbox boundaries and restrictive CSP over hidden execution paths. [35] [39] [34]

⁂

next

can you draft all of these safely:
• A signed manifest schema for hotspots and actions.
• A canvas click-handler implementation.
• A secure workflow executor API.
• An NFT-gated access flow.
• A full reference architecture diagram and folder structure.
Can u produce a concrete starter project with `frontend`, `api`, `manifest`, and `workflow` directories and the exact JSON schema for the clickable character system. A signed manifest schema for clickable image workflows.
• A canvas-based loader with hotspot detection.
• A wallet/NFT-gated workflow authorization layer.
• A reference architecture that integrates all three cleanly.
If useful, I can draft the full folder structure and API contracts for the secure canvas + NFT workflow system.

Absolutely. Here is a safe starter project that gives you the full **clickable character** system without any stealth execution inside PNGs. It uses a signed manifest, a canvas hotspot loader, a guarded API, and an optional NFT-gating layer, while keeping the image asset non-executable. [52] [53] [54]

Architecture

The clean separation is:

- `frontend/`: canvas UI and hotspot detection.
- `api/`: consent, authorization, and workflow execution.
- `manifest/`: JSON schema plus signed manifests.
- `workflow/`: allowlisted workflow handlers only.

That structure keeps behavior explicit, auditable, and browser-safe. The PNG stays an image; the manifest and API carry the logic. [53] [52]

Signed manifest schema

Use a manifest that describes the asset, hotspots, signature, and policies. Here is the exact schema shape I recommend:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/clickable-character.manifest.schema.json",
  "title": "Clickable Character Manifest",
  "type": "object",
  "required": ["version", "asset", "signature", "hotspots", "policies"],
  "properties": {
    "version": {
      "type": "string",
      "pattern": "^1\\.\\d+\\.\\d+$"
    },
    "asset": {
      "type": "object",
      "required": ["pngPath", "sha256"],
      "properties": {
        "pngPath": { "type": "string" },
        "sha256": {
          "type": "string",
          "pattern": "^[a-fA-F0-9]{64}$"
        }
      }
    },
    "signature": {
      "type": "object",
      "required": ["alg", "keyId", "value"],
      "properties": {
        "alg": {
          "type": "string",
          "enum": ["ed25519"]
        },
        "keyId": { "type": "string" },
        "value": { "type": "string" }
      }
    },
    "hotspots": {
      "type": "array",
```

```
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["id", "name", "shape", "actionId"],
        "properties": {
          "id": { "type": "string" },
          "name": { "type": "string" },
          "shape": {
            "type": "object",
            "required": ["type"],
            "properties": {
              "type": {
                "type": "string",
                "enum": ["rect", "circle", "poly"]
              },
              "x": { "type": "number" },
              "y": { "type": "number" },
              "w": { "type": "number" },
              "h": { "type": "number" },
              "r": { "type": "number" },
              "points": {
                "type": "array",
                "items": {
                  "type": "array",
                  "items": { "type": "number" },
                  "minItems": 2,
                  "maxItems": 2
                }
              }
            }
          },
          "actionId": { "type": "string" },
          "requiresNft": { "type": "boolean" },
          "nftPolicyId": { "type": "string" }
        }
      }
    },
    "policies": {
      "type": "object",
      "required": ["clickConsent", "noEmbeddedCode", "allowlistOnly"],
      "properties": {
        "clickConsent": { "type": "string" },
        "noEmbeddedCode": { "type": "boolean" },
        "allowlistOnly": { "type": "boolean" }
      }
    }
  }
}
```

Example manifest

This is a safe example manifest with one public hotspot and one NFT-gated hotspot:

```
{
  "version": "1.0.0",
  "asset": {
    "pngPath": "/assets/character.png",
    "sha256": "<sha256>"
  },
  "signature": {
    "alg": "ed25519",
    "keyId": "dev-key-1",
    "value": "<base64sig>"
  },
  "hotspots": [
    {
      "id": "eye-1",
      "name": "left-eye",
      "shape": { "type": "circle", "x": 128, "y": 96, "r": 20 },
      "actionId": "inspect",
      "requiresNft": false
    },
    {
      "id": "hand-1",
      "name": "right-hand",
      "shape": { "type": "rect", "x": 220, "y": 180, "w": 60, "h": 80 },
      "actionId": "launch_workflow",
      "requiresNft": true,
      "nftPolicyId": "premium"
    }
  ],
  "policies": {
    "clickConsent": "User must explicitly click to run an approved action.",
    "noEmbeddedCode": true,
    "allowlistOnly": true
  }
}
```

Canvas loader

The canvas loader should only render the PNG and detect clicks. It should never read or run code from the image. The loader can fetch the manifest, verify it, and dispatch only approved actions.

```
<!doctype html>
<html>
  <body>
    <canvas id="stage"></canvas>
    <script type="module" src="/app.js"></script>
  </body>
</html>
```

```
const canvas = document.getElementById('stage');
const ctx = canvas.getContext('2d');
let manifest;
const img = new Image();

async function boot() {
  manifest = await fetch('/manifest/clickable-character.manifest.example.json').then

  img.onload = () => {
    canvas.width = img.width;
    canvas.height = img.height;
    ctx.drawImage(img, 0, 0);
  };

  img.src = '/assets/character.png';
  canvas.addEventListener('click', onClick);
}

function hitTest(x, y, h) {
  if (h.shape.type === 'circle') {
    const dx = x - h.shape.x;
    const dy = y - h.shape.y;
    return dx * dx + dy * dy <= h.shape.r * h.shape.r;
  }
  if (h.shape.type === 'rect') {
    return x >= h.shape.x && x <= h.shape.x + h.shape.w && y >= h.shape.y && y <= h.s
  }
  return false;
}

async function onClick(e) {
  const rect = canvas.getBoundingClientRect();
  const x = (e.clientX - rect.left) * (canvas.width / rect.width);
  const y = (e.clientY - rect.top) * (canvas.height / rect.height);

  const hotspot = manifest.hotspots.find(h => hitTest(x, y, h));
  if (!hotspot) return;

  const consent = confirm(`Run approved action: ${hotspot.actionId}?`);
  if (!consent) return;

  const auth = await fetch('/api/authorize-action', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ hotspotId: hotspot.id, actionId: hotspot.actionId })
  }).then(r => r.json());

  if (!auth.allowed) return alert(auth.reason);

  await fetch('/api/execute-workflow', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ actionId: hotspot.actionId, context: auth.context })
  });

  alert(`Authorized: ${hotspot.actionId}`);
```

```
}

boot();
```

Secure workflow API

The API should do three things: verify consent context, enforce an allowlist, and queue execution. If NFT ownership is required, check it here or in a separate auth service. Do not accept arbitrary script text.

```
import express from 'express';

const app = express();
app.use(express.json());

const allowlist = new Set(['inspect', 'launch_workflow']);

app.post('/authorize-action', (req, res) => {
  const { actionId } = req.body;

  if (!allowlist.has(actionId)) {
    return res.json({ allowed: false, reason: 'Action not allowlisted' });
  }

  res.json({
    allowed: true,
    context: {
      session: 'ok',
      policy: 'allowlist-only'
    }
  });
});

app.post('/execute-workflow', async (req, res) => {
  const { actionId, context } = req.body;

  if (!allowlist.has(actionId)) {
    return res.status(403).json({ error: 'forbidden' });
  }

  res.json({
    ok: true,
    actionId,
    context,
    workflowStatus: 'queued'
  });
});

app.listen(3000);
```

NFT-gated access

Use NFT ownership only as an authorization condition. The NFT should unlock access to approved workflows or premium hotspots, not provide executable behavior.

Recommended pattern:

- Wallet connects.
- User signs a nonce.
- Backend verifies signature.
- Backend checks token ownership.
- Backend issues a short-lived session token.
- Session token determines whether premium hotspots are enabled.

This keeps NFT logic in the auth layer and avoids mixing it with payload execution. [55] [56]

Workflow handlers

Each workflow should be a named module in the `workflow/` directory and only exported through the registry. Example:

```
export async function inspect(ctx) {
  return { ok: true, kind: 'inspect', ctx };
}
```

```
export async function launch_workflow(ctx) {
  return { ok: true, kind: 'launch_workflow', ctx };
}
```

And a registry:

```
{
  "inspect": "./inspect.js",
  "launch_workflow": "./launch_workflow.js"
}
```

Folder structure

```
clickable-character-starter/
  frontend/
    index.html
    app.js
    csp.txt
  api/
    server.js
    README.md
  manifest/
    clickable-character.manifest.schema.json
```

```
    clickable-character.manifest.example.json
  workflow/
    inspect.js
    launch_workflow.js
    registry.json
  README.md
```

Safety notes

Use a strict CSP, sandbox any dynamic rendering, and avoid `eval()` entirely in the main app. Modern web guidance recommends restrictive CSP and sandboxed frames as core defenses against unintended script execution. If you later add NFT gating, treat it as access control only; do not let ownership metadata control code paths beyond approved workflow enablement. [54] [56] [57] [52] [53] [55]

I can also turn this into a fuller starter repo with:

- `package.json`
- `Dockerfile`
- signature verification code
- wallet login flow
- manifest validation tests
- a Mermaid architecture diagram

⁂

turn this into a fuller starter repo with:

• package.json
• Dockerfile
• signature verification code
• wallet login flow
• manifest validation tests
• a Mermaid architecture diagram

Yes — here's a fuller safe starter repo scaffold for the clickable-character system, including package metadata, Docker, signature verification, wallet login, validation tests, and a Mermaid architecture diagram. I'm keeping it explicitly non-stealthy: PNGs remain assets, manifests are signed, and workflows are allowlisted and authorized before execution. [58] [59] [60]

Repo layout

```
clickable-character-starter/
  api/
    server.js
    auth.js
    manifestVerifier.js
```

```
    nft.js
    workflowExecutor.js
    package.json
    Dockerfile
  frontend/
    index.html
    app.js
    csp.txt
  manifest/
    clickable-character.manifest.schema.json
    clickable-character.manifest.example.json
  workflow/
    inspect.js
    launch_workflow.js
    registry.json
  test/
    manifest.test.js
  docs/
    architecture.mmd
  README.md
```

api/package.json

```
{
  "name": "clickable-character-api",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "node server.js",
    "test": "node --test ../test/manifest.test.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "jose": "^5.6.3",
    "ethers": "^6.13.4",
    "ajv": "^8.17.1"
  }
}
```

api/Dockerfile

```
FROM node:20-alpine

WORKDIR /app
COPY api/package.json ./package.json
RUN npm install --omit=dev

COPY api/ .
EXPOSE 3000
CMD ["node", "server.js"]
```

Signature verification

Use Ed25519 for manifest signatures. The verifier checks the canonical manifest payload against the signed value before the client trusts hotspot data.

`api/manifestVerifier.js`

```
import { createPublicKey, verify } from 'crypto';

export function verifyManifestSignature(manifest, publicKeyPem) {
  const { signature, ...unsigned } = manifest;
  const payload = Buffer.from(JSON.stringify(unsigned));
  const pub = createPublicKey(publicKeyPem);
  return verify(null, payload, pub, Buffer.from(signature.value, 'base64'));
}
```

If you want stronger reproducibility, canonicalize JSON first with a stable serializer before signing and verifying.

Wallet login flow

Use a nonce-based sign-in flow:

1. Backend creates a nonce.
2. Frontend asks the wallet to sign it.
3. Backend verifies the signature.
4. Backend checks NFT ownership.
5. Backend returns a short-lived session token.

`api/auth.js`

```
import { verifyMessage } from 'ethers';

const nonceStore = new Map();

export function createNonce(address) {
  const nonce = crypto.randomUUID();
  nonceStore.set(address.toLowerCase(), nonce);
  return nonce;
}

export function verifyWalletLogin(address, message, signature, expectedNonce) {
  const recovered = verifyMessage(message, signature);
  if (recovered.toLowerCase() !== address.toLowerCase()) return false;
  return message.includes(expectedNonce);
}
```

`frontend/app.js` wallet login sketch

```
async function login() {
  const [address] = await window.ethereum.request({ method: 'eth_requestAccounts' })
  const nonceRes = await fetch('/api/auth/nonce', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ address })
  }).then(r => r.json());

  const message = `Sign in to Clickable Character\nNonce: ${nonceRes.nonce}`;
  const signature = await window.ethereum.request({
    method: 'personal_sign',
    params: [message, address]
  });

  const session = await fetch('/api/auth/verify', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ address, message, signature })
  }).then(r => r.json());

  return session;
}
```

NFT gating

The NFT check should only determine access. It should never supply behavior. A simple backend check can confirm token ownership or contract membership before enabling premium hotspots.

`api/nft.js`

```
import { Contract, JsonRpcProvider } from 'ethers';

export async function hasRequiredNft({ rpcUrl, contractAddress, abi, ownerAddress })
  const provider = new JsonRpcProvider(rpcUrl);
  const contract = new Contract(contractAddress, abi, provider);
  const balance = await contract.balanceOf(ownerAddress);
  return balance > 0n;
}
```

Workflow executor

Keep execution limited to a registry of known workflow modules. The API should resolve an `actionId` to a module path and invoke only that module.

`api/workflowExecutor.js`

```
import workflows from '../workflow/registry.json' assert { type: 'json' };

export async function executeWorkflow(actionId, context) {
  if (!Object.hasOwn(workflows, actionId)) {
    throw new Error('Action not allowlisted');
  }

  const mod = await import(workflows[actionId], { assert: { type: 'json' } }).catch(a
  const fn = mod[actionId];
  if (typeof fn !== 'function') throw new Error('Workflow handler missing');
  return fn(context);
}
```

Manifest validation tests

Use AJV to validate the manifest shape and a test to ensure required fields exist.

`test/manifest.test.js`

```
import test from 'node:test';
import assert from 'node:assert/strict';
import Ajv from 'ajv';
import schema from '../manifest/clickable-character.manifest.schema.json' assert { ty
import manifest from '../manifest/clickable-character.manifest.example.json' assert

test('manifest validates against schema', () => {
  const ajv = new Ajv({ allErrors: true });
  const validate = ajv.compile(schema);
  const ok = validate(manifest);
  assert.equal(ok, true, JSON.stringify(validate.errors, null, 2));
});
```

Mermaid architecture diagram

Save this as `docs/architecture.mmd`:

```
flowchart LR
  U[User] --> F[Frontend Canvas UI]
  F --> M[Signed Manifest]
  F --> A[Auth API]
  A --> N[NFT Ownership Check]
  A --> S[Session Token]
  F --> W[Workflow API]
  W --> R[Workflow Registry]
  R --> X[Allowlisted Workflow Handler]
  F -->|click hotspot| W
  M -->|hotspots, policies| F
```

That diagram follows the recommended pattern of keeping architecture documentation next to code and focusing on clear container-level boundaries. [61] [62]

Starter security defaults

- CSP: block inline scripts and external code execution.
- Canvas: render only the PNG.
- Manifest: verify signature before use.
- Actions: allowlist only.
- Wallet: nonce-sign-in flow.
- NFT: access control only.
- Logging: record every triggered action and authorization decision.

One important fix

Your frontend should never parse or execute payloads from pixel data or PNG chunks. The safe design is “image plus signed manifest plus authorized workflow,” which aligns with browser security guidance on CSP and sandboxing rather than hidden execution paths. [59] [60] [63] [58]

I can next produce the remaining concrete files in one shot:

- `server.js`
- `auth.js`
- `nft.js`
- `workflowExecutor.js`
- `frontend/app.js`
- `README.md`
- full `manifest.schema.json`
- and a complete Mermaid sequence diagram for login and click-to-action.

⁂

next produce the remaining concrete files in one shot:

• server.js
• auth.js
• nft.js
• workflowExecutor.js
• frontend/app.js
• README. md
• tull manifest. schema. json
• and a complete Mermaid sequence diagram for login and click-to-action

How to store Ed25519 public keys for manifest rotation
Implementing CSP policies to restrict dynamic script execution
→
Designing the registry.json for allowed workflow actions

Below is a safe, concrete drop-in set for the remaining files, plus the key-rotation, CSP, and workflow-registry guidance you asked for. This keeps execution explicit and allowlisted, and it does not execute anything from PNG content itself. [76] [77] [78]

`api/server.js`

```
import express from 'express';
import crypto from 'crypto';
import { createNonce, verifyWalletLogin } from './auth.js';
import { hasRequiredNft } from './nft.js';
import { verifyManifest } from './manifestVerifier.js';
import { executeWorkflow } from './workflowExecutor.js';
import manifest from '../manifest/clickable-character.manifest.example.json' assert

const app = express();
app.use(express.json());

const sessions = new Map();

const NFT_CONFIG = {
  rpcUrl: process.env.RPC_URL || '',
  contractAddress: process.env.NFT_CONTRACT_ADDRESS || '',
  abi: [
    'function balanceOf(address owner) view returns (uint256)'
  ]
};

app.get('/health', (_, res) => res.json({ ok: true }));

app.post('/api/auth/nonce', (req, res) => {
  const { address } = req.body;
  if (!address) return res.status(400).json({ error: 'address required' });
  const nonce = createNonce(address);
  res.json({ nonce });
});

app.post('/api/auth/verify', async (req, res) => {
  const { address, message, signature } = req.body;
  if (!address || !message || !signature) {
    return res.status(400).json({ error: 'address, message, signature required' });
  }

  const ok = verifyWalletLogin(address, message, signature);
  if (!ok) return res.status(401).json({ error: 'invalid signature' });

  let nftOk = false;
  try {
    nftOk = await hasRequiredNft({
      ...NFT_CONFIG,
```

```
      ownerAddress: address
    });
  } catch {
    nftOk = false;
  }

  const session = crypto.randomUUID();
  sessions.set(session, {
    address: address.toLowerCase(),
    nftOk,
    createdAt: Date.now()
  });

  res.json({ session, nftOk });
});

app.post('/api/authorize-action', async (req, res) => {
  const { session, hotspotId, actionId } = req.body;
  const s = sessions.get(session);
  if (!s) return res.status(401).json({ allowed: false, reason: 'invalid session' })

  const hotspot = manifest.hotspots.find(h => h.id === hotspotId && h.actionId === ac
  if (!hotspot) return res.json({ allowed: false, reason: 'unknown hotspot' });

  if (hotspot.requiresNft && !s.nftOk) {
    return res.json({ allowed: false, reason: 'NFT ownership required' });
  }

  const verified = verifyManifest(manifest);
  if (!verified) return res.json({ allowed: false, reason: 'manifest signature invali

  res.json({
    allowed: true,
    context: {
      address: s.address,
      hotspotId,
      actionId,
      policy: 'allowlist-only'
    }
  });
});

app.post('/api/execute-workflow', async (req, res) => {
  const { actionId, context } = req.body;
  try {
    const result = await executeWorkflow(actionId, context);
    res.json({ ok: true, result });
  } catch (err) {
    res.status(403).json({ ok: false, error: err.message });
  }
});

app.listen(process.env.PORT || 3000, () => {
  console.log('API listening');
});
```

api/auth.js

```
import { verifyMessage } from 'ethers';
import crypto from 'crypto';

const nonces = new Map();

export function createNonce(address) {
  const nonce = crypto.randomUUID();
  nonces.set(address.toLowerCase(), nonce);
  return nonce;
}

export function verifyWalletLogin(address, message, signature) {
  const recovered = verifyMessage(message, signature);
  const expectedNonce = nonces.get(address.toLowerCase());
  if (!expectedNonce) return false;
  if (recovered.toLowerCase() !== address.toLowerCase()) return false;
  return message.includes(expectedNonce);
}
```

api/nft.js

```
import { Contract, JsonRpcProvider } from 'ethers';

export async function hasRequiredNft({ rpcUrl, contractAddress, abi, ownerAddress })
  if (!rpcUrl || !contractAddress) return false;
  const provider = new JsonRpcProvider(rpcUrl);
  const contract = new Contract(contractAddress, abi, provider);
  const balance = await contract.balanceOf(ownerAddress);
  return balance > 0n;
}
```

api/workflowExecutor.js

```
import registry from '../workflow/registry.json' assert { type: 'json' };

export async function executeWorkflow(actionId, context) {
  const relPath = registry[actionId];
  if (!relPath) throw new Error('Action not allowlisted');

  const mod = await import(new URL(`../workflow/${actionId}.js`, import.meta.url));
  const fn = mod[actionId];
  if (typeof fn !== 'function') throw new Error('Workflow handler missing');
  return await fn(context);
}
```

frontend/app.js

```
const canvas = document.getElementById('stage');
const ctx = canvas.getContext('2d');
let manifest;
let session = null;
const img = new Image();

function hitTest(x, y, h) {
  if (h.shape.type === 'circle') {
    const dx = x - h.shape.x;
    const dy = y - h.shape.y;
    return dx * dx + dy * dy <= h.shape.r * h.shape.r;
  }
  if (h.shape.type === 'rect') {
    return x >= h.shape.x && x <= h.shape.x + h.shape.w && y >= h.shape.y && y <= h.s
  }
  if (h.shape.type === 'poly' && Array.isArray(h.shape.points)) {
    let inside = false;
    const pts = h.shape.points;
    for (let i = 0, j = pts.length - 1; i < pts.length; j = i++) {
      const xi = pts[i][^7_0], yi = pts[i][^7_1];
      const xj = pts[j][^7_0], yj = pts[j][^7_1];
      const intersect = ((yi > y) !== (yj > y)) && (x < (xj - xi) * (y - yi) / ((yj -
      if (intersect) inside = !inside;
    }
    return inside;
  }
  return false;
}

async function login() {
  const [address] = await window.ethereum.request({ method: 'eth_requestAccounts' })
  const nonceRes = await fetch('/api/auth/nonce', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ address })
  }).then(r => r.json());

  const message = `Sign in to Clickable Character\nNonce: ${nonceRes.nonce}`;
  const signature = await window.ethereum.request({
    method: 'personal_sign',
    params: [message, address]
  });

  session = await fetch('/api/auth/verify', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ address, message, signature })
  }).then(r => r.json());
}

async function boot() {
  manifest = await fetch('/manifest/clickable-character.manifest.example.json').then

  img.onload = () => {
```

```
    canvas.width = img.width;
    canvas.height = img.height;
    ctx.drawImage(img, 0, 0);
  };
  img.src = manifest.asset.pngPath;

  await login();

  canvas.addEventListener('click', async (e) => {
    const rect = canvas.getBoundingClientRect();
    const x = (e.clientX - rect.left) * (canvas.width / rect.width);
    const y = (e.clientY - rect.top) * (canvas.height / rect.height);

    const hotspot = manifest.hotspots.find(h => hitTest(x, y, h));
    if (!hotspot) return;

    const consent = confirm(`Run approved action: ${hotspot.actionId}?`);
    if (!consent) return;

    const auth = await fetch('/api/authorize-action', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        session: session.session,
        hotspotId: hotspot.id,
        actionId: hotspot.actionId
      })
    }).then(r => r.json());

    if (!auth.allowed) return alert(auth.reason);

    const result = await fetch('/api/execute-workflow', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        actionId: hotspot.actionId,
        context: auth.context
      })
    }).then(r => r.json());

    alert(JSON.stringify(result, null, 2));
  });
}

boot();
```

manifest/clickable-character.manifest.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/clickable-character.manifest.schema.json",
  "title": "Clickable Character Manifest",
  "type": "object",
  "required": ["version", "asset", "signature", "hotspots", "policies"],
  "properties": {
```

```
    "version": {
      "type": "string",
      "pattern": "^1\\.\\d+\\.\\d+$"
    },
    "asset": {
      "type": "object",
      "required": ["pngPath", "sha256"],
      "properties": {
        "pngPath": { "type": "string" },
        "sha256": {
          "type": "string",
          "pattern": "^[a-fA-F0-9]{64}$"
        }
      }
    },
    "signature": {
      "type": "object",
      "required": ["alg", "keyId", "value"],
      "properties": {
        "alg": {
          "type": "string",
          "enum": ["ed25519"]
        },
        "keyId": { "type": "string" },
        "value": { "type": "string" }
      }
    },
    "hotspots": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["id", "name", "shape", "actionId"],
        "properties": {
          "id": { "type": "string" },
          "name": { "type": "string" },
          "shape": {
            "type": "object",
            "required": ["type"],
            "properties": {
              "type": {
                "type": "string",
                "enum": ["rect", "circle", "poly"]
              },
              "x": { "type": "number" },
              "y": { "type": "number" },
              "w": { "type": "number" },
              "h": { "type": "number" },
              "r": { "type": "number" },
              "points": {
                "type": "array",
                "items": {
                  "type": "array",
                  "items": { "type": "number" },
                  "minItems": 2,
                  "maxItems": 2
```

```
                }
              }
            }
          },
          "actionId": { "type": "string" },
          "requiresNft": { "type": "boolean" },
          "nftPolicyId": { "type": "string" }
        }
      }
    },
    "policies": {
      "type": "object",
      "required": ["clickConsent", "noEmbeddedCode", "allowlistOnly"],
      "properties": {
        "clickConsent": { "type": "string" },
        "noEmbeddedCode": { "type": "boolean" },
        "allowlistOnly": { "type": "boolean" },
        "csp": { "type": "string" }
      }
    }
  }
}
```

workflow/registry.json

```
{
  "inspect": "./inspect.js",
  "launch_workflow": "./launch_workflow.js"
}
```

README.md

```
# Clickable Character Starter

Safe architecture for image-driven workflows.

## What this is
A canvas-based UI that loads a PNG image, verifies a signed manifest, checks wallet/N

## What this is not
This does not execute code from PNG files, embedded chunks, or hidden payloads.

## Setup
1. Install dependencies in `api/`.
2. Set `RPC_URL` and `NFT_CONTRACT_ADDRESS` if using NFT gating.
3. Start the API on port 3000.
4. Serve `frontend/` and `manifest/` from the same origin or a dev proxy.

## Security model
- PNG is asset-only.
- Manifest is signed with Ed25519.
- Wallet login uses nonce + signature.
- NFT gating is access control only.
```

```
- Workflow execution is allowlisted.
- CSP blocks dynamic execution in the browser.

## Endpoints
- POST /api/auth/nonce
- POST /api/auth/verify
- POST /api/authorize-action
- POST /api/execute-workflow

## Notes on key rotation
Store multiple Ed25519 public keys in a trust set keyed by `kid`. During rotation, pu

## CSP
Use a strict policy with no inline script execution, no `eval`, and no remote script

## Workflow registry
Only action IDs listed in `workflow/registry.json` may run. That file is the single s
```

Key rotation design

For Ed25519 public keys, store them as a versioned trust list with `kid`, `alg`, `publicKey`, `notBefore`, and `notAfter`. Keep at least two keys during rotation: the current signing key and the next key, then overlap them for a grace period. This is the same general principle used in other secure signing systems: deploy the new public key before switching signing, then retire the old key after consumers have updated. [79] [80]

A simple trust-store shape:

```
{
  "keys": [
    {
      "kid": "key-2026-07",
      "alg": "Ed25519",
      "publicKey": "BASE64...",
      "notBefore": "2026-07-01T00:00:00Z",
      "notAfter": "2026-10-01T00:00:00Z"
    }
  ]
}
```

CSP policy guidance

For this kind of app, prefer a strict CSP that:

- blocks inline scripts,
- forbids `unsafe-eval`,
- sets `object-src 'none'`,
- sets `base-uri 'none'`,
- keeps `frame-ancestors 'none'`,

- and uses nonces or hashes for scripts if needed. Modern strict CSP guidance recommends generating a fresh nonce per page load and binding scripts to that nonce rather than relying on broad source whitelists. Be careful with `'strict-dynamic'`; it extends trust to scripts loaded by already trusted scripts, so it should be used only when you fully control those code paths. [81] [77] [82] [76]

A practical header example:

```
Content-Security-Policy:
default-src 'self';
script-src 'nonce-{RANDOM}' 'strict-dynamic';
object-src 'none';
base-uri 'none';
frame-ancestors 'none';
connect-src 'self' https:;
img-src 'self' data:;
style-src 'self';
```

Mermaid sequence diagram

Save this as `docs/login-and-click.mmd`:

```
sequenceDiagram
  participant U as User
  participant F as Frontend
  participant A as API
  participant W as Wallet
  participant N as NFT Check
  participant R as Workflow Runtime

  U->>F: Open app
  F->>A: Request nonce
  A-->>F: Nonce
  F->>W: Sign nonce
  W-->>F: Signature
  F->>A: Verify login
  A->>N: Check NFT ownership
  N-->>A: Ownership result
  A-->>F: Session token + nftOk

  U->>F: Click hotspot
  F->>A: Authorize action
  A-->>F: Allowed / denied
  F->>A: Execute workflow
  A->>R: Call allowlisted handler
  R-->>A: Result
  A-->>F: Workflow result
```

CSP and workflow registry files

If you want, I can produce the exact missing helper files next:

- `api/manifestVerifier.js`
- `workflow/inspect.js`
- `workflow/launch_workflow.js`
- `frontend/index.html`
- `frontend/csp.txt`
- a signed-manifest signing script for local development

⁂

produce the exact missing helper files next:

• api/manifestVerifier.js
• workflow/inspect.js
• workflow/launch_workflow.js
• frontend/index.html
• frontend/csp.txt
• a signed-manifest signing script for local development
How to structure the signed-manifest signing script for local dev

Implementing CSP policies for image-based workflow injectors

Best practices for auditing the workflowExecutor
for vulnerabilities

Yes — and the inventory you shared is useful here because it already flags that character records should include provenance, runtime details, and explicit evidence gates before any publish/mint action. That aligns well with a safe workflow registry, manifest verification, and an execution path that fails closed if anything is missing or untrusted. [98] [99]

Missing helper files

`api/manifestVerifier.js`

```
import { createPublicKey, verify } from 'crypto';

function canonicalize(value) {
  if (Array.isArray(value)) return `[${value.map(canonicalize).join(',')}]`;
  if (value && typeof value === 'object') {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ':' + canonica
  }
  return JSON.stringify(value);
}

export function verifyManifest(manifest, publicKeyPem = process.env.MANIFEST_PUBLIC_
```

```
  if (!manifest || !manifest.signature || !publicKeyPem) return false;
  const { signature, ...unsigned } = manifest;
  const payload = Buffer.from(canonicalize(unsigned));
  const publicKey = createPublicKey(publicKeyPem);
  return verify(
    null,
    payload,
    publicKey,
    Buffer.from(signature.value, 'base64')
  );
}
```

workflow/inspect.js

```
export async function inspect(context) {
  return {
    ok: true,
    action: 'inspect',
    summary: 'Approved inspection workflow executed.',
    context
  };
}

export default inspect;
```

workflow/launch_workflow.js

```
export async function launch_workflow(context) {
  return {
    ok: true,
    action: 'launch_workflow',
    summary: 'Approved workflow launch executed.',
    context
  };
}

export default launch_workflow;
```

frontend/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width,initial-scale=1" />
    <title>Clickable Character</title>
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-sr
  </head>
  <body>
    <div id="status">Loading…</div>
    <canvas id="stage"></canvas>
```

```
    <pre id="output"></pre>
    <button id="connect-wallet">Connect Wallet</button>
    <script type="module" src="/app.js"></script>
  </body>
</html>
```

frontend/csp.txt

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; ba
```

Local signing script

This is a safe local-dev script that signs the manifest file, not any image payload. It canonicalizes JSON, hashes it, signs with Ed25519, and writes out a signed manifest.

scripts/sign-manifest.mjs

```
import fs from 'fs';
import crypto from 'crypto';

function canonicalize(value) {
  if (Array.isArray(value)) return `[${value.map(canonicalize).join(',')}]`;
  if (value && typeof value === 'object') {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ':' + canonical
  }
  return JSON.stringify(value);
}

const [, , inFile, outFile, privateKeyFile, keyId = 'dev-key-1'] = process.argv;

if (!inFile || !outFile || !privateKeyFile) {
  console.error('Usage: node scripts/sign-manifest.mjs <in.json> <out.json> <private
  process.exit(1);
}

const manifest = JSON.parse(fs.readFileSync(inFile, 'utf8'));
delete manifest.signature;

const payload = canonicalize(manifest);
const privateKeyPem = fs.readFileSync(privateKeyFile, 'utf8');
const sig = crypto.sign(null, Buffer.from(payload), privateKeyPem).toString('base64'

const signed = {
  ...manifest,
  signature: {
    alg: 'ed25519',
    keyId,
    value: sig
  }
};
```

```
fs.writeFileSync(outFile, JSON.stringify(signed, null, 2));
console.log(`Wrote signed manifest to ${outFile}`);
```

How to structure local dev signing

- Keep one private key per environment.
- Store public keys in the app trust list by `kid`.
- Sign only the manifest JSON.
- Never sign runtime code or image bytes.
- Rotate keys by publishing a new `kid` before switching signatures; keep the old `kid` valid during a short overlap window.[100]

CSP guidance

For an image-driven workflow UI, CSP should assume the PNG is inert and only the app logic is active. Keep `script-src` self-only or nonce-based, avoid `unsafe-inline`, avoid `unsafe-eval`, and do not allow arbitrary remote scripts. Strict CSP guidance recommends nonce or hash-based script approval and minimizing script origins; browser docs also recommend `object-src 'none'` and `base-uri 'none'`.[101] [102] [103]

A good rule for this project is:

- frontend scripts: local bundle only.
- API calls: same-origin or explicitly trusted API origin.
- images: self or data URIs only.
- no dynamic script injection.
- no eval-based workflow loading.

Workflow registry best practice

Design `workflow/registry.json` as the only allowlist of executable actions. The registry should map simple action IDs to local modules, and the API should reject anything not present in the registry. That aligns with the character inventory's "no merge, deployment, publication, billing action, or mint without evidence" style gating, which is basically a policy allowlist pattern.[99]

Example:

```
{
  "inspect": "./inspect.js",
  "launch_workflow": "./launch_workflow.js"
}
```

Recommended audit checks for `workflowExecutor`:

- Reject action IDs not in registry.

- Disallow path traversal in module resolution.
- Enforce a timeout per workflow.
- Log action ID, session, and manifest hash.
- Verify workflow outputs are JSON-safe.
- Keep execution isolated from request parsing.
- Add tests for unauthorized action, missing module, timeout, and malformed output.

Mermaid sequence diagram

```
sequenceDiagram
  participant U as User
  participant F as Frontend
  participant A as API
  participant W as Wallet
  participant M as Manifest Verifier
  participant N as NFT Check
  participant X as Workflow Executor

  U->>F: Open app
  F->>M: Load + verify signed manifest
  M-->>F: Validated hotspots/policies

  U->>F: Connect wallet
  F->>W: Request signature over nonce
  W-->>F: Signature
  F->>A: POST /api/auth/verify
  A->>N: Check NFT ownership
  N-->>A: ownership result
  A-->>F: session token

  U->>F: Click hotspot
  F->>A: POST /api/authorize-action
  A-->>F: allowed / denied
  F->>A: POST /api/execute-workflow
  A->>X: Resolve allowlisted workflow
  X-->>A: result
  A-->>F: workflow result
```

Audit checklist for `workflowExecutor`

Use this checklist when reviewing the executor for vulnerabilities:

- Only import from local allowlisted files.
- No user-controlled file paths.
- No `eval`, `Function`, or runtime code generation.
- Enforce request and execution timeouts.
- Make output schema strict.
- Fail closed on module load errors.

- Log rejected action attempts.
- Test NFT-gated and non-gated paths separately.
- Confirm CSP prevents browser-side code injection.[102] [103] [104]

One useful note from your files

Your inventory repeatedly says records should not be merged, published, billed, or minted without evidence, and that declared image, metadata, animation, 3D, audio, and AR assets must be provenance-checked first. That is a good operational model for this repo: treat signed manifests and workflow allowlists as the evidence gate, and keep the PNG purely as the visual character asset.[105] [98] [99]

I can next assemble these into a single consolidated repo tree with all files in one code block, or produce a `package.json` + `Dockerfile` that matches these exact modules.

⁂

next assemble these into a single consolidated repo tree with all files in one code block, & produce a package.json + Dockerfile that matches these exact modules.

Here's the consolidated repo tree plus a matching `package.json` and `Dockerfile` for the exact modules we assembled. The layout keeps PNGs asset-only, uses signed manifests, and routes every action through an allowlisted workflow registry rather than any embedded code path.[106] [107] [108]

Repo tree

```
clickable-character-starter/
├── api/
│   ├── auth.js
│   ├── manifestVerifier.js
│   ├── nft.js
│   ├── server.js
│   ├── workflowExecutor.js
│   └── package.json
├── docs/
│   ├── architecture.mmd
│   └── login-and-click.mmd
├── frontend/
│   ├── app.js
│   ├── csp.txt
│   └── index.html
├── manifest/
│   ├── clickable-character.manifest.example.json
│   └── clickable-character.manifest.schema.json
├── scripts/
```

```
│   └── sign-manifest.mjs
├── test/
│   └── manifest.test.js
├── workflow/
│   ├── inspect.js
│   ├── launch_workflow.js
│   └── registry.json
└── README.md
```

api/package.json

```
{
  "name": "clickable-character-api",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "node server.js",
    "start": "node server.js",
    "test": "node --test ../test/manifest.test.js"
  },
  "dependencies": {
    "ajv": "^8.17.1",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "ethers": "^6.13.4",
    "express": "^4.19.2",
    "helmet": "^8.0.0",
    "jose": "^5.6.3"
  }
}
```

api/Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY api/package.json ./package.json
RUN npm install --omit=dev

COPY api/ .
COPY manifest/ ./manifest/
COPY workflow/ ./workflow/
COPY scripts/ ./scripts/
COPY test/ ./test/
COPY README.md ./README.md

ENV NODE_ENV=production
EXPOSE 3000

CMD ["node", "server.js"]
```

Notes on the Dockerfile

This is a simple runtime image for the exact module set we built. For a stricter production build, you could add a non-root user and separate build/runtime stages, but this version matches the current plain JS repo layout cleanly. [109] [110] [111]

How the pieces fit

- `api/server.js` exposes auth, manifest verification, authorization, and workflow execution.
- `api/auth.js` handles nonce-based wallet login.
- `api/nft.js` checks ownership as an access-control signal.
- `api/manifestVerifier.js` validates signed manifests with Ed25519.
- `api/workflowExecutor.js` resolves only allowlisted workflow actions.
- `frontend/app.js` renders the image on canvas and calls the API.
- `scripts/sign-manifest.mjs` signs manifests for local development.
- `workflow/registry.json` is the allowlist source of truth.

That structure mirrors the safer “asset + signed manifest + controlled execution” model rather than any payload-in-image pattern, which is important given the CSP and signing boundaries involved. [112] [107] [106]

One implementation caution

If you later tighten this for production, add a nonce-based CSP, remove any `connect-src` wildcards, and keep `script-src` restricted to self or nonced scripts only. For signed documents, keep the Ed25519 trust roots versioned by `kid` and rotate them with overlap instead of hard cutovers. [108] [113] [112]

I can next produce the full `README.md` and a one-command bootstrap script that installs dependencies, generates a dev Ed25519 keypair, and signs the sample manifest.

⁂

next produce the full README.md and a one-command bootstrap script that installs dependencies, generates a dev Ed25519 keypair, and signs the sample manifest. Build a hardened repo structure: consolidated file tree, secure package.json, and production-ready...

Computer
How to automate manifest signing using the Ed25519 scripts
Best practices for securing the workflow registry.json file

Below is the full README and a one-command bootstrap script, plus a hardened structure and the recommended secure handling for manifest signing and the workflow registry. The design keeps the image inert, keeps the manifest signed, and keeps execution inside allowlisted workflow modules only.[124] [125] [126]

Full `README.md`

````
# Clickable Character Starter

A safe, signed-manifest, canvas-driven workflow starter for clickable image interacti

This repo intentionally does **not** execute code from PNG files or image chunks. The

## Goals

- Render a PNG character on an HTML5 canvas.
- Detect clicks on predefined hotspots.
- Require wallet login before privileged actions.
- Gate optional premium actions with NFT ownership.
- Verify signed manifests before use.
- Execute only allowlisted workflows.
- Keep the system auditable and non-stealthy.

## Repo layout

```text
clickable-character-starter/
├── api/
│   ├── auth.js
│   ├── manifestVerifier.js
│   ├── nft.js
│   ├── server.js
│   ├── workflowExecutor.js
│   ├── package.json
│   └── Dockerfile
├── docs/
│   ├── architecture.mmd
│   └── login-and-click.mmd
````
````

```
├── frontend/
│   ├── app.js
│   ├── csp.txt
│   └── index.html
├── manifest/
│   ├── clickable-character.manifest.example.json
│   └── clickable-character.manifest.schema.json
├── scripts/
│   ├── bootstrap-dev.sh
│   └── sign-manifest.mjs
├── test/
│   └── manifest.test.js
├── workflow/
│   ├── inspect.js
│   ├── launch_workflow.js
│   └── registry.json
└── README.md
```

Runtime model

1. The frontend fetches the sample manifest.
2. The manifest is verified against the Ed25519 public key.
3. The user logs in with a wallet signature over a nonce.
4. The backend checks NFT ownership if a hotspot requires it.
5. The frontend sends an allowlisted action request.
6. The backend resolves the action in `workflow/registry.json`.
7. The workflow module executes in a controlled server-side path.

What is signed

Only the manifest JSON is signed. The signature covers the canonical JSON payload and is verified before the UI trusts hotspot data.

What is not signed

- PNG bytes are not executed.
- No hidden code is loaded from image metadata or chunks.
- No workflow code is embedded in the asset.
- No arbitrary JavaScript is accepted from the client.

Prerequisites

- Node.js 20+
- npm 9+
- Optional: MetaMask or another EVM wallet for login
- Optional: RPC URL and NFT contract address for NFT gating

Install

```
cd api
npm install
```

Local development bootstrap

Run the bootstrap script from the repo root:

```
bash scripts/bootstrap-dev.sh
```

It will:

- install dependencies,
- generate a dev Ed25519 keypair,
- write the private/public keys to `secrets/`,
- sign the sample manifest,
- and print the generated public key fingerprint.

Environment variables

Create a `.env` file in `api/` or export these variables:

```
PORT=3000
MANIFEST_PUBLIC_KEY=-----BEGIN PUBLIC KEY-----
...
-----END PUBLIC KEY-----
RPC_URL=https://...
NFT_CONTRACT_ADDRESS=0x...
FRONTEND_URL=http://localhost:8080
```

Development

Start the API:

```
cd api
npm run dev
```

Serve the frontend from the repo root with any static server, or use your preferred dev toolchain.

Manifest validation

Run tests:

```
cd api
npm test
```

Signed manifest workflow

Sign a manifest

```
node scripts/sign-manifest.mjs \
  manifest/clickable-character.manifest.example.json \
  manifest/clickable-character.manifest.signed.json \
  secrets/dev-ed25519-private.pem \
  dev-key-1
```

Verify a manifest

The backend verifies the manifest signature before exposing interactive hotspots.

Wallet login

The wallet login flow is nonce-based:

1. Client requests a nonce.
2. Client signs a message with the wallet.
3. Server verifies the recovered address.
4. Server creates a short-lived session.
5. Session can be used to authorize hotspot actions.

NFT gating

NFT ownership is used only as an authorization signal. It controls whether a premium hotspot may run; it never provides executable code.

CSP policy

Use a strict CSP. The frontend should avoid inline scripts, `unsafe-eval`, and third-party script injection.

Recommended policy:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; ba
```

Workflow registry

`workflow/registry.json` is the allowlist. Only actions in that file can be executed.

Example:

```
{
  "inspect": "./inspect.js",
  "launch_workflow": "./launch_workflow.js"
}
```

Auditing checklist

- Reject unknown action IDs.
- Reject path traversal in workflow resolution.
- Enforce execution timeouts.
- Validate workflow output shape.
- Log rejected requests.
- Fail closed on manifest verification errors.
- Keep private keys out of source control.
- Rotate keys with overlapping trust windows.

Key rotation

Use versioned `kid` values for public keys, keep the old key valid during a short overlap, then retire it after all deployed clients have moved to the new key.

Production hardening

For production, add:

- non-root Docker user,
- rate limiting,
- structured logging,
- secret management,
- a database-backed session store,
- a proper reverse proxy,
- and CI checks for manifest schema and signature validation.

Mermaid diagrams

See:

- docs/architecture.mmd
- docs/login-and-click.mmd

```
## One-command bootstrap script

Save as `scripts/bootstrap-dev.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[^10_0]}")/.." && pwd)"
SECRET_DIR="$ROOT_DIR/secrets"
MANIFEST_IN="$ROOT_DIR/manifest/clickable-character.manifest.example.json"
MANIFEST_OUT="$ROOT_DIR/manifest/clickable-character.manifest.signed.json"
SIGNER="$ROOT_DIR/scripts/sign-manifest.mjs"

mkdir -p "$SECRET_DIR"

echo "Installing API dependencies..."
(cd "$ROOT_DIR/api" && npm install)

if [ ! -f "$SECRET_DIR/dev-ed25519-private.pem" ]; then
  echo "Generating dev Ed25519 keypair..."
  node - <<'NODE'
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');

const root = process.argv[^10_1];
const secretDir = path.join(root, 'secrets');
fs.mkdirSync(secretDir, { recursive: true });

const { privateKey, publicKey } = crypto.generateKeyPairSync('ed25519');
fs.writeFileSync(
  path.join(secretDir, 'dev-ed25519-private.pem'),
  privateKey.export({ format: 'pem', type: 'pkcs8' })
);
fs.writeFileSync(
  path.join(secretDir, 'dev-ed25519-public.pem'),
  publicKey.export({ format: 'pem', type: 'spki' })
);
NODE
fi

echo "Signing sample manifest..."
node "$SIGNER" \
  "$MANIFEST_IN" \
  "$MANIFEST_OUT" \
  "$SECRET_DIR/dev-ed25519-private.pem" \
```
```

```
  "dev-key-1"

echo "Done."
echo "Public key:"
cat "$SECRET_DIR/dev-ed25519-public.pem"
```

Make it executable:

```
chmod +x scripts/bootstrap-dev.sh
```

Hardened repo structure

A production-ready arrangement should keep secrets and build artifacts outside source, and keep the runtime surface small:

```
clickable-character-starter/
├── api/
├── frontend/
├── manifest/
├── workflow/
├── scripts/
├── docs/
├── test/
├── secrets/                # gitignored
├── .env                     # gitignored
├── .gitignore
├── README.md
└── package.json             # optional root workspace file
```

For a stricter setup, you can add a root workspace `package.json` and keep all install logic in one place. That is usually easier to audit than multiple ad hoc dependency trees, and it helps reduce supply-chain ambiguity in npm-based projects. [127] [128] [129] [130]

Secure `package.json`

If you want a root workspace version, use this:

```
{
  "name": "clickable-character-starter",
  "private": true,
  "version": "1.0.0",
  "workspaces": [
    "api"
  ],
  "scripts": {
    "bootstrap": "bash scripts/bootstrap-dev.sh",
    "test": "npm --workspace api test",
    "dev": "npm --workspace api run dev"
  }
}
```

Best practices for `workflow/registry.json`

Treat it like a policy file, not just a lookup table. Keep it tiny, committed, reviewed, and generated only through code review or CI so it does not become a hidden execution surface. Recommended controls:

- only relative local module paths,
- no user-editable runtime paths,
- no URL imports,
- no globbing,
- no dynamic `eval`,
- no environment-driven path concatenation,
- and no direct client influence over action IDs beyond the allowlist. [129] [131] [132] [133]

Automating manifest signing

The clean automation pattern is:

1. Generate the canonical payload.
2. Sign with the local dev private key.
3. Write the signed manifest to a separate file.
4. Verify the signed manifest in CI using the public key.
5. Publish only the signed artifact.

That gives you a reproducible signed-manifest pipeline and keeps the trust root separate from the app. Ed25519 generation and verification can be done with Node's built-in crypto primitives, which support Ed25519 key generation and signing directly. [134] [135] [136]

CSP reminder

Use a strict policy and do not relax it for workflow convenience. The frontend should never need to execute code from the manifest or the image, and browser guidance recommends nonce/hash-based script authorization with `object-src 'none'` and `base-uri 'none'`. [125] [126] [124]

Next step

I can now produce the exact `package.json` and `Dockerfile` for the root workspace version, plus the full `bootstrap-dev.sh` and `sign-manifest.mjs` files in one consolidated code block.

⁂

Next step

I can now produce the exact package.json and Dockerfile for the root workspace version, plus the full bootstrap-dev.sh and sign-manifest.mjs files in one consolidated code block. How to implement Ed25519 signature validation in the api/manifestVerifier.js module
Best practices for managing the registry.json workflow allowlist in production environments
How to harden the Dockerfile for a production image-payload service deployment

Use the manifest verifier as a strict trust gate: canonicalize the unsigned manifest deterministically, verify the Ed25519 signature against the correct public key, and fail closed on any parse, canonicalization, or key mismatch. In production, keep the workflow registry tiny and immutable by deployment, and harden the Docker image with a pinned base image, non-root execution, multi-stage build boundaries, and a health check. [146] [147] [148] [149]

Ed25519 verifier

Here is the implementation pattern for `api/manifestVerifier.js`:

```
import { createPublicKey, verify } from 'crypto';

function canonicalize(value) {
  if (Array.isArray(value)) {
    return `[${value.map(canonicalize).join(',')}]`;
  }
  if (value && typeof value === 'object') {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ':' + canonical
  }
  return JSON.stringify(value);
}

export function verifyManifest(manifest, publicKeyPem = process.env.MANIFEST_PUBLIC_K
  if (!manifest || !manifest.signature || !publicKeyPem) return false;

  const { signature, ...unsigned } = manifest;
  const payload = Buffer.from(canonicalize(unsigned));
  const publicKey = createPublicKey(publicKeyPem);

  return verify(null, payload, publicKey, Buffer.from(signature.value, 'base64'));
}
```

This follows the safe pattern of verifying a canonicalized payload with a published public key, which is the same general model used by independent manifest verification systems that rely on Ed25519 plus canonical JSON. [147] [146]

Workflow registry

Treat `workflow/registry.json` as a deployment-controlled allowlist, not a runtime-editable configuration file. The safest production pattern is:

- keep only relative local module paths,

- reject unknown action IDs,
- avoid environment-driven path assembly,
- block path traversal,
- keep registry changes behind code review,
- and deploy the registry together with the workflow modules so the allowlist and implementation stay in sync. [150] [151] [152] [153]

A good operational rule is that the registry should be small enough that you can review every action in one glance. If you need per-tenant behavior, put it in authorization data, not in dynamic module resolution.

Hardened Dockerfile

For production, use a multi-stage build, pin the Node image, run as non-root, and include a health check. A hardened version for this repo would look like:

```
FROM node:20.11.1-alpine3.19 AS base
WORKDIR /app
ENV NODE_ENV=production

FROM base AS deps
COPY api/package.json ./package.json
RUN npm install --omit=dev && npm cache clean --force

FROM base AS runtime
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=deps /app/node_modules ./node_modules
COPY api/ ./api/
COPY manifest/ ./manifest/
COPY workflow/ ./workflow/
COPY scripts/ ./scripts/
COPY test/ ./test/
COPY README.md ./README.md
USER appuser
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 CMD node -e "
CMD ["node", "api/server.js"]
```

That aligns with modern production guidance: pin versions, separate build/runtime layers, avoid root, and keep image contents minimal. [148] [149] [154]

Production checks

Add these checks before deployment:

- confirm `MANIFEST_PUBLIC_KEY` is set from a secret manager, not source control.
- verify the signed manifest in CI with the same verifier used in production.
- run a registry test that ensures every action ID maps to a local module.
- scan the final image for CVEs.

- ensure the CSP blocks inline script execution and dynamic code loading. [155] [156] [157] [158]

One-command bootstrap direction

The bootstrap flow should:

1. install dependencies,
2. generate a dev Ed25519 keypair,
3. sign the sample manifest,
4. print the public key,
5. and leave the private key in `secrets/` which is gitignored.

That keeps local development simple while preserving the production separation between trusted signing keys and deployable app code. [159] [160] [161]

If you want, I can now output the exact final consolidated code block for:

- `package.json`
- hardened `Dockerfile`
- `scripts/bootstrap-dev.sh`
- `scripts/sign-manifest.mjs`
- `.gitignore`
- and the final `manifestVerifier.js` in one ready-to-paste bundle.

⁂

now output the exact final consolidated code block for:

• package.json
• hardened Dockerfile
• scripts/bootstrap-dev.sh
• scripts/sign-manifest.mjs
• .gitignore
• and the final manifestVerifier.js in one ready-to-paste bundle. Yes, please provide that complete bundle in one block now
Actually, can you separate the Dockerfile and bootstrap script instead
I need you to add environment variable validation to those files first

Absolutely — below is a **ready-to-paste** bundle with env validation added up front, plus the hardened Dockerfile and bootstrap split out separately as requested. I've kept the runtime model the same: signed manifest, allowlisted workflows, and no code execution from PNGs or image chunks. [172] [173] [174]

`package.json`

```
{
  "name": "clickable-character-starter",
  "private": true,
  "version": "1.0.0",
  "workspaces": [
    "api"
  ],
  "scripts": {
    "bootstrap": "bash scripts/bootstrap-dev.sh",
    "dev": "npm --workspace api run dev",
    "test": "npm --workspace api test"
  }
}
```

Hardened `Dockerfile`

```
FROM node:20.11.1-alpine3.19 AS base
WORKDIR /app
ENV NODE_ENV=production

FROM base AS deps
COPY api/package.json ./api/package.json
RUN cd api && npm install --omit=dev && npm cache clean --force

FROM base AS runtime
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=deps /app/api/node_modules ./api/node_modules
COPY api/ ./api/
COPY manifest/ ./manifest/
COPY workflow/ ./workflow/
COPY scripts/ ./scripts/
COPY test/ ./test/
COPY README.md ./README.md
USER appuser
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 CMD node -e "
CMD ["node", "api/server.js"]
```

`scripts/bootstrap-dev.sh`

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[^12_0]}")/.." && pwd)"
SECRET_DIR="$ROOT_DIR/secrets"
MANIFEST_IN="$ROOT_DIR/manifest/clickable-character.manifest.example.json"
MANIFEST_OUT="$ROOT_DIR/manifest/clickable-character.manifest.signed.json"
SIGNER="$ROOT_DIR/scripts/sign-manifest.mjs"

require_cmd() {
```

```
  command -v "$1" >/dev/null 2>&1 || { echo "Missing required command: $1" >&2; exit
}

require_cmd node
require_cmd npm

mkdir -p "$SECRET_DIR"

echo "Installing dependencies..."
(cd "$ROOT_DIR/api" && npm install)

if [ ! -f "$SECRET_DIR/dev-ed25519-private.pem" ] || [ ! -f "$SECRET_DIR/dev-ed25519-
  echo "Generating dev Ed25519 keypair..."
  node - "$ROOT_DIR" <<'NODE'
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');

const root = process.argv[^12_2];
const secretDir = path.join(root, 'secrets');
fs.mkdirSync(secretDir, { recursive: true });

const { privateKey, publicKey } = crypto.generateKeyPairSync('ed25519');
fs.writeFileSync(
  path.join(secretDir, 'dev-ed25519-private.pem'),
  privateKey.export({ format: 'pem', type: 'pkcs8' })
);
fs.writeFileSync(
  path.join(secretDir, 'dev-ed25519-public.pem'),
  publicKey.export({ format: 'pem', type: 'spki' })
);
NODE
fi

echo "Signing sample manifest..."
node "$SIGNER" \
  "$MANIFEST_IN" \
  "$MANIFEST_OUT" \
  "$SECRET_DIR/dev-ed25519-private.pem" \
  "dev-key-1"

echo "Done."
echo "Public key:"
cat "$SECRET_DIR/dev-ed25519-public.pem"
```

scripts/sign-manifest.mjs

```
import fs from 'fs';
import crypto from 'crypto';

function canonicalize(value) {
  if (Array.isArray(value)) return `[${value.map(canonicalize).join(',')}]`;
  if (value && typeof value === 'object') {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ':' + canonical
  }
```

```
  return JSON.stringify(value);
}

const [, , inFile, outFile, privateKeyFile, keyId = 'dev-key-1'] = process.argv;

if (!inFile || !outFile || !privateKeyFile) {
  console.error('Usage: node scripts/sign-manifest.mjs <in.json> <out.json> <private
  process.exit(1);
}

const manifest = JSON.parse(fs.readFileSync(inFile, 'utf8'));
delete manifest.signature;

const payload = canonicalize(manifest);
const privateKeyPem = fs.readFileSync(privateKeyFile, 'utf8');
const sig = crypto.sign(null, Buffer.from(payload), privateKeyPem).toString('base64')

const signed = {
  ...manifest,
  signature: {
    alg: 'ed25519',
    keyId,
    value: sig
  }
};

fs.writeFileSync(outFile, JSON.stringify(signed, null, 2));
console.log(`Wrote signed manifest to ${outFile}`);
```

.gitignore

```
node_modules/
api/node_modules/
dist/
build/
coverage/
.env
.env.*
secrets/
manifest/*.signed.json
*.log
.DS_Store
```

`api/manifestVerifier.js`

```
import { createPublicKey, verify } from 'crypto';

function canonicalize(value) {
  if (Array.isArray(value)) return `[${value.map(canonicalize).join(',')}]`;
  if (value && typeof value === 'object') {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ':' + canonical
  }
  return JSON.stringify(value);
```

```
}

export function verifyManifest(manifest, publicKeyPem = process.env.MANIFEST_PUBLIC_K
  if (!manifest || typeof manifest !== 'object') return false;
  if (!manifest.signature || !manifest.signature.value || !publicKeyPem) return false
  if (manifest.signature.alg !== 'ed25519') return false;

  const { signature, ...unsigned } = manifest;
  const payload = Buffer.from(canonicalize(unsigned));
  const publicKey = createPublicKey(publicKeyPem);

  try {
    return verify(null, payload, publicKey, Buffer.from(signature.value, 'base64'));
  } catch {
    return false;
  }
}
```

Env validation additions

To make this bundle actually boot safely, add env validation early in `api/server.js` or a new config file before any other imports are used. A minimal version is:

```
const required = ['MANIFEST_PUBLIC_KEY'];
for (const key of required) {
  if (!process.env[key]) {
    throw new Error(`Missing required environment variable: ${key}`);
  }
}
```

If you want stricter production-grade validation, use a schema at startup and fail fast when anything is missing or malformed. That approach is standard for Node services and prevents “booted with broken config” failures.[175] [176] [177]

Production hardening notes

For the Docker image, the main production protections are: pinned base image, non-root user, minimal copied artifacts, health check, and no secret material baked into layers. For the workflow allowlist, keep it static, local, reviewed, and deployment-controlled so action resolution can’t be influenced at runtime.[178] [179] [180] [181]

One practical reminder

If you later want, I can produce the matching `api/server.js` startup config block with proper environment schema validation wired in, so `MANIFEST_PUBLIC_KEY`, `RPC_URL`, and `NFT_CONTRACT_ADDRESS` are all validated before the server starts.

⁂

can u produce the matching `api/server.js` startup config block with proper environment schema validation wired in, so `MANIFEST_PUBLIC_KEY`, `RPC_URL`, and `NFT_CONTRACT_ADDRESS` are all validated before the server starts.

Integrate Ed25519 signature verification into api/server.js
Explain the allowlist registry security logic in the workflow module
Add unit tests for the manifest canonicalization and signing flow

#!/usr/bin/env python3
"""Upload MiseOS catalog assets to DigitalOcean Spaces.

Dry-run is the default. Set --apply only after environment variables are configured.
Required:
SPACES_KEY
SPACES_SECRET
SPACES_REGION
SPACES_BUCKET
Optional:
SPACES_ENDPOINT (defaults to https://{region}.digitaloceanspaces.com)
SPACES_CDN_BASE (public base URL, e.g. https://cdn.miseos.dev)
"""
from pathlib import Path
import argparse, mimetypes, os, sys
try:
import boto3
except ImportError:
raise SystemExit("Install boto3: python -m pip install boto3")

ROOT = Path(__file__).resolve().parents[1]
parser = argparse.ArgumentParser()
parser.add_argument("--apply", action="store_true")
parser.add_argument("--prefix", default="miseos/v1")
args = parser.parse_args()

region=os.environ.get("SPACES_REGION","")
bucket=os.environ.get("SPACES_BUCKET","")
endpoint=os.environ.get("SPACES_ENDPOINT") or (f"https://{region}.digitaloceanspaces.com" if region else "")
required=["SPACES_KEY","SPACES_SECRET","SPACES_REGION","SPACES_BUCKET"]
missing=[x for x in required if not os.environ.get(x)]
if args.apply and missing:
raise SystemExit("Missing environment variables: "+", ".join(missing))

```
files=[]
for relroot in
["metadata","assets/images","assets/reference","assets/thumbnails","media","catalog","taxonomy","
registry"]:
for p in (ROOT/relroot).rglob("*"):
if p.is_file() and not p.name.endswith(".slot.json"):
files.append(p)

for p in files:
key=f"{args.prefix}/{p.relative_to(ROOT).as_posix()}"
ctype=mimetypes.guess_type(p.name)[0] or "application/octet-stream"
print(("UPLOAD" if args.apply else "DRY-RUN"), p.relative_to(ROOT), "->", key, ctype)

if not args.apply:
print(f"Dry-run complete: {len(files)} files. Re-run with --apply to upload.")
sys.exit(0)

client=boto3.client("s3", region_name=region, endpoint_url=endpoint,
aws_access_key_id=os.environ["SPACES_KEY"],
aws_secret_access_key=os.environ["SPACES_SECRET"])
for p in files:
key=f"{args.prefix}/{p.relative_to(ROOT).as_posix()}"
ctype=mimetypes.guess_type(p.name)[0] or "application/octet-stream"
client.upload_file(str(p), bucket, key, ExtraArgs={"ACL":"public-
read","ContentType":ctype,"CacheControl":"public,max-age=31536000,immutable"})
print(f"Uploaded {len(files)} files to s3://{bucket}/{args.prefix}/")

from pathlib import Path
import csv, json, re, hashlib, zipfile, shutil
from collections import Counter, defaultdict

from artifact_tool import Workbook, SpreadsheetFile

ROOT = Path("/mnt/data")
PDF_PATH = ROOT / "0ACE220D-32A1-49F3-A00C-69E4A6197ACC Compressed(1).pdf"
OUT = ROOT / "miseos_full_roster_taxonomy"
if OUT.exists():
shutil.rmtree(OUT)
(OUT / "data").mkdir(parents=True)
(OUT / "taxonomy").mkdir()
(OUT / "registry").mkdir()
(OUT / "photo-alignment").mkdir()
(OUT / "thumbnails").mkdir()
```

Canonical 92 records supplied in the user's roster text.

roster_tsv = """1\tAlko Commit Chef\tCommit Intake\tGenesis\tPipeline Kitchen\tcharacters.generated.json
2\tBruno Blast-Radius\tChange Impact Scanner\tRare\tPipeline Kitchen\tcharacters.generated.json
3\tLila Branchkeeper\tBranch Policy Gate\tRare\tPipeline Kitchen\tcharacters.generated.json
4\tOmar Reviewer\tPR Hygiene Check\tGenesis\tPipeline Kitchen\tcharacters.generated.json
5\tHugo Matrix\tTest Matrix Runner\tCommon\tPipeline Kitchen\tcharacters.generated.json
6\tSeira Scan\tSecurity Scan Stage\tRare\tPipeline Kitchen\tcharacters.generated.json
7\tYume Builder\tArtifact Builder\tGenesis\tPipeline Kitchen\tcharacters.generated.json
8\tIron Hashi\tArtifact Validator\tGenesis\tPipeline Kitchen\tcharacters.generated.json
9\tNina Promoter\tEnvironment Promotion Gate\tCommon\tPipeline Kitchen\tcharacters.generated.json
10\tRelease Conductor\tDeployment Orchestrator\tCommon\tPipeline Kitchen\tcharacters.generated.json
11\tFelix Rollback\tRollback & Fix-Forward\tCommon\tPipeline Kitchen\tcharacters.generated.json
12\tTariq Notesmith\tRelease Notes Generator\tCommon\tPipeline Kitchen\tcharacters.generated.json
13\tIvun Cache\tPipeline Cost Monitor\tCommon\tPipeline Kitchen\tcharacters.generated.json
14\tCanary Taster\tPipeline Observability\tRare\tPipeline Kitchen\tcharacters.generated.json
15\tPipeline Sensei\tPipeline Tuner Agent\tEpic\tPipeline Kitchen\tcharacters.generated.json
16\tTemplate Tanaka\tService Template Catalog\tCommon\tPlatform Kitchen\tcharacters.generated.json
17\tBootstrap Aiko\tNew Service Wizard\tGenesis\tPlatform Kitchen\tcharacters.generated.json
18\tTerra Forge\tEnvironment Provisioner\tCommon\tPlatform Kitchen\tcharacters.generated.json
19\tSchema Sora\tDatabase Blueprint Manager\tCommon\tPlatform Kitchen\tcharacters.generated.json
20\tRuntime Riku\tRuntime Profile Manager\tCommon\tPlatform Kitchen\tcharacters.generated.json
21\tVault Hana\tSecrets & Config Broker\tRare\tPlatform Kitchen\tcharacters.generated.json
22\tPolicy Kenji\tPolicy Packs\tRare\tPlatform Kitchen\tcharacters.generated.json
23\tGrafana Mei\tShared Observability Pack\tCommon\tPlatform Kitchen\tcharacters.generated.json
24\tUmami Code\tTraffic & Ingress Manager\tGenesis\tPlatform Kitchen\tcharacters.generated.json
25\tQueue Mizu\tJob & Queue Templates\tGenesis\tPlatform Kitchen\tcharacters.generated.json
26\tStreama\tData Streaming Profiles\tGenesis\tPlatform Kitchen\tcharacters.generated.json
27\tPreview Yuki\tPreview Environment Spawner\tCommon\tPlatform Kitchen\tcharacters.generated.json
28\tPortal Rei\tSelf-Service Portal UI\tCommon\tPlatform Kitchen\tcharacters.generated.json
29\tGuardrail Goro\tGolden Path Guardrails\tCommon\tPlatform Kitchen\tcharacters.generated.json
30\tPlatform Sensei\tPlatform Health Board\tEpic\tPlatform Kitchen\tcharacters.generated.json

31\tSLO Sage\tSLO Definition Workflow\tGenesis\tReliability
Kitchen\tcharacters.generated.json
32\tTrace Sensei\tInstrumentation Booster\tRare\tReliability
Kitchen\tcharacters.generated.json
33\tChaos Kaito\tLoad & Chaos Profiles\tCommon\tReliability
Kitchen\tcharacters.generated.json
34\tCapacity Kanna\tPre-Deployment Reliability Check\tCommon\tReliability
Kitchen\tcharacters.generated.json
35\tCanary Judge\tCanary Judge\tRare\tReliability Kitchen\tcharacters.generated.json
36\tAlert Akira\tIncident Detection Rules\tCommon\tReliability
Kitchen\tcharacters.generated.json
37\tPager Hiro\tIncident Triage Runbook\tCommon\tReliability
Kitchen\tcharacters.generated.json
38\tOnCall Naomi\tOn-Call Rotation Manager\tCommon\tReliability
Kitchen\tcharacters.generated.json
39\tReview Ren\tPost-Incident Review\tCommon\tReliability
Kitchen\tcharacters.generated.json
40\tBacklog Benji\tReliability Backlog Builder\tCommon\tReliability
Kitchen\tcharacters.generated.json
41\tMetrics Miko\tSLO Dashboards\tGenesis\tReliability Kitchen\tcharacters.generated.json
42\tForecast Fuji\tCapacity Planner\tCommon\tReliability Kitchen\tcharacters.generated.json
43\tResilience Ryu\tResilience Testing Loop\tRare\tReliability
Kitchen\tcharacters.generated.json
44\tImpact Ichi\tCustomer Impact Simulator\tCommon\tReliability
Kitchen\tcharacters.generated.json
45\tReliability Sensei\tReliability Advisor Agent\tEpic\tReliability
Kitchen\tcharacters.generated.json
46\tThreat Taro\tThreat Modeling Starter\tCommon\tSecurity
Kitchen\tcharacters.generated.json
47\tSecure Sakura\tSecure Template Library\tCommon\tSecurity
Kitchen\tcharacters.generated.json
48\tToken Toshi\tCredential Hygiene Scanner\tRare\tSecurity
Kitchen\tcharacters.generated.json
49\tCVE Kumo\tDependency Risk Monitor\tRare\tSecurity Kitchen\tcharacters.generated.json
50\tContainer Kage\tContainer Hardening Workflow\tCommon\tSecurity
Kitchen\tcharacters.generated.json
51\tSeira Vaultkeeper\tSecrets Lifecycle Manager\tGenesis\tSecurity
Kitchen\tcharacters.generated.json
52\tPolicy Ronin\tPolicy-as-Code Engine\tGenesis\tSecurity Kitchen\tcharacters.generated.json
53\tMerge Sentinel\tPre-Merge Security Gate\tRare\tSecurity
Kitchen\tcharacters.generated.json
54\tWatchtower Yoru\tRuntime Threat Detector\tCommon\tSecurity
Kitchen\tcharacters.generated.json
55\tAuditor Emi\tAccess Review Workflow\tCommon\tSecurity
Kitchen\tcharacters.generated.json
56\tIncident Kitsune\tSecurity Incident Playbooks\tRare\tSecurity

Kitchen\tcharacters.generated.json
57\tEvidence Scribe\tCompliance Evidence Collector\tRare\tSecurity
Kitchen\tcharacters.generated.json
58\tNudge Neko\tSecurity Training Nudges\tRare\tSecurity Kitchen\tcharacters.generated.json
59\tScorecard Shin\tSecurity Scorecard\tRare\tSecurity Kitchen\tcharacters.generated.json
60\tRed Oni\tRed Team / Chaos Security Mode\tEpic\tSecurity
Kitchen\tcharacters.generated.json
61\tTaxonomy Taka\tEvent Taxonomy Manager\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
62\tLake Haru\tTelemetry Ingestion Pipeline\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
63\tLattice\tTrace Correlator\tGenesis\tData & Insight Kitchen\tcharacters.generated.json
64\tRepo Ranger\tRepo Health Analyzer\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
65\tDORA Daichi\tPipeline Analytics Engine\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
66\tFlag Fox\tFeature Flag Analytics\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
67\tCost Kaito\tCost & Efficiency Monitor\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
68\tDX Mei\tDeveloper Experience Survey Loop\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
69\tRevenue Rina\tCustomer Behavior → Incident Linker\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
70\tRisk Ryota\tRisk Scoring Model\tRare\tData & Insight Kitchen\tcharacters.generated.json
71\tApprentice Kai\tSelf-Improvement Agent\tGenesis\tData & Insight
Kitchen\tcharacters.generated.json
72\tAnomaly Aki\tAnomaly Detection Studio\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
73\tLeaderboard Leo\tScorecards & Leaderboards\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
74\tStoryteller Sumi\tNarrative Report Generator\tCommon\tData & Insight
Kitchen\tcharacters.generated.json
75\tRoadmap Sensei\tStrategic Roadmap Recommender\tGenesis\tData & Insight
Kitchen\tcharacters.generated.json
76\tKurogami Senpai\tOrchestrate\tMythic\tOperations Kitchen\tmise_orchestra_cards.csv
77\tTensho\tTask Flow\tRare\tPipeline Kitchen\tmise_orchestra_cards.csv
78\tMizu\tPrepare\tRare\tOperations Kitchen\tmise_orchestra_cards.csv
79\tKaji\tCompute\tEpic\tOperations Kitchen\tmise_orchestra_cards.csv
80\tYume\tBuild\tRare\tPipeline Kitchen\tmise_orchestra_cards.csv
81\tNull-chan\tCleanse\tRare\tReliability Kitchen\tmise_orchestra_cards.csv
82\tFlash Riku\tDeploy\tEpic\tPipeline Kitchen\tmise_orchestra_cards.csv
83\tSeira\tSecure\tEpic\tPlatform Kitchen\tmise_orchestra_cards.csv
84\tKuro Guard\tProtect\tEpic\tPlatform Kitchen\tmise_orchestra_cards.csv
85\tScale Oni\tScale\tLegendary\tOperations Kitchen\tmise_orchestra_cards.csv
86\tBuild-Kun\tBuild\tRare\tPipeline Kitchen\tmise_orchestra_cards.csv

87\tOrchestra Core\tCoordinate\tMythic\tPlatform Kitchen\tmise_orchestra_cards.csv
88\tData-chan\tCollect\tRare\tData & Insight Kitchen\tmise_orchestra_cards.csv
89\tInsight Sensei\tAnalyze\tEpic\tData & Insight Kitchen\tmise_orchestra_cards.csv
90\tChaos Analyst\tLearn\tLegendary\tData & Insight Kitchen\tmise_orchestra_cards.csv
91\tGrowth Guru\tOptimize\tEpic\tData & Insight Kitchen\tmise_orchestra_cards.csv
92\tCulture Chef\tGrow\tLegendary\tData & Insight Kitchen\tmise_orchestra_cards.csv"""

base_records = []
for line in roster_tsv.splitlines():
n, name, role, rarity, dept, source = line.split("\t")
base_records.append({
"number": int(n), "name": name, "agent_function": role,
"rarity": rarity, "department": dept, "source": source,
"record_origin": "user_supplied_roster"
})

20 additional photo-aligned cards chosen from distinct PDF cards to complete 112.
photo_expansion = [
(93, "Velvet Vesper", "Guest Concierge", "Rare", 9),
(94, "Flavor Forager", "Ingredient Scout", "Rare", 10),
(95, "Crate Courier", "Provision Scout", "Common", 11),
(96, "Ledger Ladle", "Cost Steward", "Epic", 12),
(97, "Beacon Broil", "Alert Dispatcher", "Epic", 13),
(98, "Ladle Lumen", "Broth Beacon", "Rare", 14),
(99, "Reserve Rune", "Backup Archivist", "Epic", 15),
(100, "Vacuum Vesper", "Seal Keeper", "Rare", 16),
(101, "Flavor Scribe", "Menu Curator", "Epic", 17),
(102, "Prep Pulse", "Mise Monitor", "Rare", 18),
(103, "Queue Quartz", "Service Timer", "Epic", 19),
(104, "Harvest Hex", "Produce Alchemist", "Legendary", 20),
(105, "Garnish Ghost", "Finisher", "Legendary", 21),
(106, "Pantry Paladin", "Provision Steward", "Epic", 22),
(107, "Ticket Tactician", "Order Strategist", "Rare", 23),
(108, "Plate Inspector", "Asset Auditor", "Epic", 24),
(109, "Timer Tactician", "Chrono Chef", "Epic", 25),
(110, "Recipe Ranger", "Knowledge Pathfinder", "Rare", 26),
(111, "Portion Paladin", "Balance Steward", "Epic", 27),
(112, "Allergen Oracle", "Safety Seer", "Legendary", 28),
]
for n, name, role, rarity, page in photo_expansion:
base_records.append({
"number": n, "name": name, "agent_function": role,
"rarity": rarity, "department": "Operations Kitchen",
"source": f"uploaded_pdf_page_{page}",
"record_origin": "photo_aligned_completion",
"primary_photo_page": page
})

```
assert len(base_records) 112

def slugify(text):
return re.sub(r"[^a-z0-9]+", "-", text.lower()).strip("-")

dept_codes = {
"Pipeline Kitchen": "pipeline",
"Platform Kitchen": "platform",
"Reliability Kitchen": "reliability",
"Security Kitchen": "security",
"Data & Insight Kitchen": "data-insight",
"Operations Kitchen": "operations",
}
dept_missions = {
"Pipeline Kitchen": "Build. Test. Deploy. Repeat.",
"Platform Kitchen": "Provide golden paths and stable service foundations.",
"Reliability Kitchen": "Observe. Measure. Improve.",
"Security Kitchen": "Guard. Audit. Comply.",
"Data & Insight Kitchen": "Learn. Optimize. Evolve.",
"Operations Kitchen": "Keep the kitchen running smoothly.",
}
dept_colors = {
"Pipeline Kitchen": "#E97822",
"Platform Kitchen": "#1B2A41",
"Reliability Kitchen": "#2E6B4E",
"Security Kitchen": "#8B1E2D",
"Data & Insight Kitchen": "#17A2A4",
"Operations Kitchen": "#D4AF37",
}
rarity_tiers = {"Common": 1, "Rare": 2, "Epic": 3, "Legendary": 4, "Genesis": 5, "Mythic": 6}

Derive complete character metadata deterministically.
characters = []
by_dept = defaultdict(list)
for rec in base_records:
by_dept[rec["department"]].append(rec)

for rec in base_records:
num = rec["number"]
name = rec["name"]
slug = slugify(name)
rarity = rec["rarity"]
tier = rarity_tiers[rarity]
seed = int(hashlib.sha256(f"{num}:{name}".encode()).hexdigest()[:8], 16)
stats = {
"power": 55 + (seed % 26) + tier * 2,
"precision": 58 + ((seed >> 3) % 25) + tier * 2,
```

```
"speed": 52 + ((seed >> 6) % 28) + tier,
"resilience": 60 + ((seed >> 9) % 22) + tier * 2,
"intelligence": 57 + ((seed >> 12) % 25) + tier * 2,
"teamwork": 62 + ((seed >> 15) % 21) + tier * 2,
}
stats = {k: min(99, v) for k, v in stats.items()}
dept_peers = [x["name"] for x in by_dept[rec["department"]] if x["name"] != name]
idx = [x["name"] for x in by_dept[rec["department"]]].index(name)
handoffs = [dept_peers[(idx + j) % len(dept_peers)] for j in range(min(3, len(dept_peers)))] if
dept_peers else []
char = {
"id": f"miseos.character.{num:03d}",
"number": num,
"slug": slug,
"name": name,
"mise_name": name,
"codename": slug.replace("-", "_").upper(),
"record_kind": "character_card",
"collection": "MiseOS Character Agents",
"department": rec["department"],
"department_code": dept_codes[rec["department"]],
"department_mission": dept_missions[rec["department"]],
"agent_function": rec["agent_function"],
"job_name": rec["agent_function"],
"rarity": rarity,
"rarity_tier": tier,
"source": rec["source"],
"record_origin": rec["record_origin"],
"visual": {
"primary_color": dept_colors[rec["department"]],
"card_style": "black-gold holo DevOps orchestra",
"aura": f"{rec['department']} operational aura",
"tool_symbol": rec["agent_function"],
"avatar_path": f"assets/characters/{slug}/avatar.png",
"card_path": f"assets/characters/{slug}/card.png",
"primary_photo_page": rec.get("primary_photo_page")
},
"workflow": {
"summary": f"{name} performs {rec['agent_function'].lower()} within the {rec['department']}.",
"inputs": ["repository context", "task request", "policy constraints", "current evidence"],
"outputs": ["validated result", "evidence record", "handoff packet"],
"synced_agents": handoffs,
"proof_gate": "No merge, deployment, publication, billing action, or mint without evidence."
},
"stats": stats,
"paths": {
```

```
"readme": f"characters/{slug}/README.md",
"metadata": f"characters/{slug}/metadata.json"
},
"status": "canonical" if rec["record_origin"] "user_supplied_roster" else
"photo_aligned_canonical_completion",
"confidence": "high" if rec["record_origin"] "user_supplied_roster" else "medium"
}
characters.append(char)
```

Canonical 23 registry manifests.

```
registry_items = [
("miseos.agent-sdk","MiseOS Agent SDK","sdk","@mise-os/agent-sdk"),
("miseos.analytics-engine","MiseOS Analytics Engine","analytics","@mise-os/analytics-engine"),
("miseos.attestation-gate","MiseOS Attestation Gate","policy","@mise-os/attestation-gate"),
("miseos.cli","MiseOS CLI","package","@mise-os/cli"),
("miseos.docs-builder","MiseOS Docs Builder","container","ghcr.io/mise-os/miseos-docs-builder"),
("miseos.docs-hub-template","MiseOS Docs Hub Template","template","@mise-os/docs-hub-
template; templates/docs-hub"),
("miseos.gradle-plugin","MiseOS Gradle Plugin","package","dev.miseos.registry-plugin"),
("miseos.maven-policy-adapter","MiseOS Maven Policy Adapter","package","dev.miseos:policy-
adapter"),
("miseos.nuget-client","MiseOS NuGet Client","sdk","MiseOS.Registry.Client"),
("miseos.policy-checker","MiseOS Policy Checker","container","
ghcr.io/mise-os/miseos-policy-checker"),
("miseos.policy-engine","MiseOS Policy Engine","policy","@mise-os/policy-engine;
ghcr.io/mise-os/miseos-policy-checker"),
("miseos.registry-schema","MiseOS Registry Schema","schema","@mise-os/registry-schema"),
("miseos.release-sentinel","MiseOS Release Sentinel","agent","@mise-os/release-sentinel"),
("miseos.repo-steward","MiseOS Repo Steward","agent","@mise-os/repo-steward"),
("miseos.ruby-steward","MiseOS Ruby Steward","package","miseos-ruby-steward"),
("miseos.runner","MiseOS Runner","container","ghcr.io/mise-os/miseos-runner"),
("miseos.sbom-indexer","MiseOS SBOM Indexer","agent","@mise-os/sbom-indexer"),
("miseos.trust-graph","MiseOS Trust Graph","analytics","@mise-os/trust-graph"),
("miseos.workflow-linter","MiseOS Workflow Linter","agent","@mise-os/workflow-linter"),
("miseos.workflow-pack","MiseOS Workflow Pack","workflow","@mise-os/workflow-pack"),
("miseos.repo-pet","MiseOS Repo Pet","agent","@mise-os/repo-pet"),
("miseos.repo-reliability-kit","MiseOS Repo Reliability Kit","workflow","@mise-os/repo-
reliability-kit"),
("miseos.orchestra-theme","Mise Orchestra Theme","theme","@mise-os/orchestra-theme"),
]
registry = []
for i, (rid, name, typ, artifact) in enumerate(registry_items, 1):
registry.append({
"catalog_number": i,
"registry_id": rid,
```

```
"name": name,
"record_kind": "registry_manifest",
"type": typ,
"version": "0.2.0",
"status": "experimental",
"artifact": artifact,
"collection": "MiseOS Registry Manifests",
"is_character": False,
"marketplace_language": "product infrastructure",
"card_language": "registry manifest badge",
"capabilities": [typ, "repo-safe", "evidence-aware"],
})
```

Photo page alignment from the uploaded 105-page PDF.
Titles are transcribed from the rendered cards.

```
page_titles = [
"MiseOS Repo Steward","MiseOS Workflow Linter","MiseOS Ruby Steward","MiseOS SBOM Indexer",
"MiseOS Workflow Pack","MiseOS Policy Engine","MiseOS Registry Schema","MiseOS Release Sentinel",
"Velvet Vesper","Flavor Forager","Crate Courier","Ledger Ladle","Beacon Broil","Ladle Lumen",
"Reserve Rune","Vacuum Vesper","Flavor Scribe","Prep Pulse","Queue Quartz","Harvest Hex",
"Garnish Ghost","Pantry Paladin","Ticket Tactician","Plate Inspector","Timer Tactician",
"Recipe Ranger","Portion Paladin","Allergen Oracle","Legacy Keeper","Menu Maestro","Supply Scout",
"Energy Keeper","Sanitation Star","Plate Inspector","Orderly Owl","Shift Sentinel","Kitchen Conductor",
"Sear Samurai","Oil Oracle","Herb Whisperer","Veg Virtuoso","Pulse Captain","Grain Guardian",
"Slicer Swift","Ashen One","Roast Commander","Brine Baron","Sweet Architect","Cacao Alchemist",
"Kombu Keeper","Flavor Hacker","Temp Tamer","Broth Scholar","Citrus Zest","Miso Maven",
"Umami Seeker","Glaze Whisperer","Crisp Master","Stock Sage","Plating Poet","No-Waste Ninja",
"Fermenta","Pickle Doc","Smoke Monk","Cold Zero","Butcher Blade","Zen Plater","Pan Phantom",
"Sauce Sage","Steam Whisper","Knead Guardian","Spice Alchemist","Byte","Char Coach",
"DORA Daichi","Cost Kaito","Anomaly Aki","Token Toshi","Scorecard Shin","Nudge Neko",
"Merge Sentinel","Incident Kitsune","Evidence Scribe","Merge Sentinel","Scorecard Shin",
"Evidence Scribe","Nudge Neko","Incident Kitsune","Token Toshi","CVE Kumo","CVE Kumo",
"Schema Sora","Runtime Riku","Preview Yuki","Portal Rei","Guardrail Goro","Grafana Mei",
"Runtime Riku","Preview Yuki","Schema Sora","Guardrail Goro","Grafana Mei","Portal Rei",
"Tariq Notesmith","Tariq Notesmith"
]
assert len(page_titles) 105

char_by_name = {c["name"].lower(): c for c in characters}
reg_by_name = {r["name"].lower(): r for r in registry}
```

```
Alias image titles for registry pages.
reg_alias = {
"miseos repo steward": "miseos.repo-steward",
"miseos workflow linter": "miseos.workflow-linter",
"miseos ruby steward": "miseos.ruby-steward",
"miseos sbom indexer": "miseos.sbom-indexer",
"miseos workflow pack": "miseos.workflow-pack",
"miseos policy engine": "miseos.policy-engine",
"miseos registry schema": "miseos.registry-schema",
"miseos release sentinel": "miseos.release-sentinel",
}
registry_by_id = {r["registry_id"]: r for r in registry}

title_counts = Counter(page_titles)
seen_counts = Counter()
alignment = []
for page, title in enumerate(page_titles, 1):
seen_counts[title] += 1
key = title.lower()
matched = None
if key in reg_alias:
r = registry_by_id[reg_alias[key]]
matched = ("registry_manifest", r["registry_id"], r["name"])
elif key in char_by_name:
c = char_by_name[key]
matched = ("character_card", c["id"], c["name"])
if matched:
kind, record_id, canonical_name = matched
status = "matched"
confidence = "high"
else:
kind, record_id, canonical_name = "unmapped_photo_card", "", title
status = "visual_reference_only"
confidence = "medium"
alignment.append({
"page": page,
"display_title": title,
"record_kind": kind,
"matched_record_id": record_id,
"canonical_name": canonical_name,
"alignment_status": status,
"confidence": confidence,
"duplicate_occurrence": seen_counts[title],
"duplicate_total": title_counts[title],
"thumbnail": f"thumbnails/page-{page:03d}.jpg",
```

```
"source_file": PDF_PATH.name,
})
```

Extract low-resolution reference thumbnails.

```
import fitz
doc = fitz.open(PDF_PATH)
for page_index in range(len(doc)):
page = doc.load_page(page_index)
pix = page.get_pixmap(matrix=fitz.Matrix(0.5, 0.5), alpha=False)
pix.save(str(OUT / "thumbnails" / f"page-{page_index+1:03d}.jpg"))
doc.close()
```

Taxonomy definitions.

```
taxonomy = {
"schema": "miseos.taxonomy.v1",
"official_model": {
"character_agents": 112,
"registry_manifests": 23,
"total_ecosystem_items": 135,
"canonical_language": "112 character agents + 23 registry manifests = 135 MiseOS ecosystem items.",
"prohibited_language": ["135 characters", "registry manifests are character cards"]
},
"record_kinds": {
"character_card": "Collectible mascot/operator with role, lore, workflow, stats, and handoffs.",
"registry_manifest": "Product infrastructure such as SDKs, packages, containers, policies, schemas, workflows, analytics, and themes.",
"unmapped_photo_card": "Visual card present in the uploaded PDF but not selected into the canonical 112 roster."
},
"departments": [
{"code": dept_codes[d], "name": d, "mission": dept_missions[d], "color": dept_colors[d]}
for d in dept_codes
],
"rarities": [
{"name": name, "tier": tier, "meaning": f"Rarity tier {tier}"}
for name, tier in rarity_tiers.items()
],
"registry_types": sorted(set(r["type"] for r in registry)),
"brand_tokens": {
"Umami Black": "#0E0F10",
"Gold Ladle": "#D4AF37",
"Broth Cream": "#F6F1E7",
```

"Flame Orange": "#E97822",
"Ink Blue": "#1B2A41",
"Matcha Green": "#2E6B4E",
},
"provenance_policy": {
"high": "Directly supplied in the roster text or exact visual title match.",
"medium": "Aligned from a visible card title and promoted to complete the official 112 count.",
"visual_reference_only": "Retained in photo alignment but not asserted as part of the canonical 112."
}
}

Write JSON.

```
ecosystem = {
"schema": "miseos.ecosystem.roster.v1",
"counts": {"character_agents": 112, "registry_manifests": 23, "total_ecosystem_items": 135},
"characters": characters,
"registry_manifests": registry,
}
(OUT / "data" / "characters.json").write_text(json.dumps(characters, indent=2,
ensure_ascii=False) + "\n")
(OUT / "registry" / "registry-manifests.json").write_text(json.dumps(registry, indent=2,
ensure_ascii=False) + "\n")
(OUT / "taxonomy" / "taxonomy.json").write_text(json.dumps(taxonomy, indent=2,
ensure_ascii=False) + "\n")
(OUT / "photo-alignment" / "photo-alignment.json").write_text(json.dumps(alignment,
indent=2, ensure_ascii=False) + "\n")
(OUT / "data" / "miseos-ecosystem-135.json").write_text(json.dumps(ecosystem, indent=2,
ensure_ascii=False) + "\n")
```

Write CSV.

```
char_fields = [
"id","number","slug","name","mise_name","codename","record_kind","collection",
"department","department_code","department_mission","agent_function","job_name",
"rarity","rarity_tier","source","record_origin","status","confidence",
"primary_color","card_style","primary_photo_page","workflow_summary","proof_gate",
"synced_agents","power","precision","speed","resilience","intelligence","teamwork",
"readme_path","metadata_path"
]
with (OUT / "data" / "characters.csv").open("w", newline="", encoding="utf-8-sig") as f:
w = csv.DictWriter(f, fieldnames=char_fields)
w.writeheader()
for c in characters:
```

```
w.writerow({
**{k: c.get(k, "") for k in char_fields},
"primary_color": c["visual"]["primary_color"],
"card_style": c["visual"]["card_style"],
"primary_photo_page": c["visual"].get("primary_photo_page") or "",
"workflow_summary": c["workflow"]["summary"],
"proof_gate": c["workflow"]["proof_gate"],
"synced_agents": " | ".join(c["workflow"]["synced_agents"]),
**c["stats"],
"readme_path": c["paths"]["readme"],
"metadata_path": c["paths"]["metadata"],
})

reg_fields =
["catalog_number","registry_id","name","record_kind","type","version","status","artifact","collectio
n","marketplace_language","card_language","capabilities"]
with (OUT / "registry" / "registry-manifests.csv").open("w", newline="", encoding="utf-8-sig") as f:
w = csv.DictWriter(f, fieldnames=reg_fields)
w.writeheader()
for r in registry:
row = r.copy()
row["capabilities"] = " | ".join(r["capabilities"])
w.writerow({k: row.get(k, "") for k in reg_fields})

align_fields = list(alignment[0].keys())
with (OUT / "photo-alignment" / "photo-alignment.csv").open("w", newline="", encoding="utf-8-
sig") as f:
w = csv.DictWriter(f, fieldnames=align_fields)
w.writeheader()
w.writerows(alignment)

Taxonomy CSV
tax_rows = []
for d in taxonomy["departments"]:
tax_rows.append({"taxonomy_group":"department","key":d["code"],"label":d["name"],"value":d["
mission"],"color":d["color"]})
for r in taxonomy["rarities"]:
tax_rows.append({"taxonomy_group":"rarity","key":str(r["tier"]),"label":r["name"],"value":r["mea
ning"],"color":""})
for t in taxonomy["registry_types"]:
tax_rows.append({"taxonomy_group":"registry_type","key":t,"label":t.title(),"value":"Registry
infrastructure category","color":""})
with (OUT / "taxonomy" / "taxonomy.csv").open("w", newline="", encoding="utf-8-sig") as f:
w = csv.DictWriter(f, fieldnames=["taxonomy_group","key","label","value","color"])
w.writeheader()
w.writerows(tax_rows)
```

README.

dept_counts = Counter(c["department"] for c in characters)
rarity_counts = Counter(c["rarity"] for c in characters)
mapped_count = sum(1 for a in alignment if a["alignment_status"] == "matched")
readme = f"""# MiseOS Full Roster and Taxonomy Metadata

112 character agents + 23 registry manifests = 135 MiseOS ecosystem items.

This package separates the collectible character roster from product infrastructure.

Contents

```
-       data/characters.json and data/characters.csv - canonical 112-character roste
-       registry/registry-manifests.json and .csv - canonical 23-item product catalo
-       data/miseos-ecosystem-135.json - combined ecosystem envelope with separate a
-       taxonomy/taxonomy.json and .csv - record kinds, departments, rarities, regis
-       photo-alignment/photo-alignment.json and .csv - all 105 uploaded PDF pages a
-       thumbnails/ - low-resolution page references used for visual alignment.
-       miseos_full_roster_taxonomy.xlsx - workbook view of all metadata.
```

Provenance

```
-       92 character records are directly transcribed from the supplied roster text.
-       20 visually distinct cards from PDF pages 9-28 complete the canonical 112-cha
-       The PDF contains repeated cards and additional visual-only concepts; those a
-       Registry manifests remain product infrastructure and are never counted as cha
```

Counts

Departments

{chr(10).join(f"- {k}: {v}" for k,v in sorted(dept_counts.items()))}

Rarities

{chr(10).join(f"- {k}: {v}" for k,v in sorted(rarity_counts.items()))}

Photo alignment

```
-       PDF pages: 105
-       Pages matched to a canonical character or registry manifest: {mapped_count}
-       Visual-reference-only pages: {105 - mapped_count}
-       """
-       (OUT / "README.md").write_text(readme, encoding="utf-8")
```

Excel workbook with artifact_tool.

```
wb = Workbook.create()
summary = wb.worksheets.add("Summary")
char_sheet = wb.worksheets.add("Character Roster")
reg_sheet = wb.worksheets.add("Registry Manifests")
photo_sheet = wb.worksheets.add("Photo Alignment")
tax_sheet = wb.worksheets.add("Taxonomy")

header_fmt = {
"fill": "#0E0F10",
"font": {"bold": True, "color": "#F6F1E7"},
"horizontal_alignment": "center",
"vertical_alignment": "center",
}
title_fmt = {
"fill": "#D4AF37",
"font": {"bold": True, "color": "#0E0F10", "size": 16},
"horizontal_alignment": "center",
"vertical_alignment": "center",
}

Summary
summary.merge_cells("A1:F1")
summary.get_range("A1").values = [["MiseOS Full Roster & Taxonomy"]]
summary.get_range("A1:F1").format = title_fmt
summary.get_range("A3:B7").values = [
["Metric","Count"],
["Character agents",112],
["Registry manifests",23],
["Total ecosystem items","=B4+B5"],
["PDF reference pages",105],
]
summary.get_range("A3:B3").format = header_fmt
summary.get_range("A9:C15").values = [["Department","Code","Character Count"]] + [
[d, dept_codes[d], dept_counts[d]] for d in dept_codes
]
summary.get_range("A9:C9").format = header_fmt
summary.get_range("E3:F10").values = [["Rarity","Count"]] + [[r, rarity_counts[r]] for r in
rarity_tiers]
summary.get_range("E3:F3").format = header_fmt
summary.get_range("A17:F20").values = [
["Canonical language","","","","","",""],
["112 character agents + 23 registry manifests = 135 MiseOS ecosystem items.","","","","","",""],
["Registry manifests are product infrastructure, not character cards.","","","","","",""],
["Photo-derived completions are explicitly marked with medium confidence.","","","","","",""],
]
```

```
summary.merge_cells("A18:F18"); summary.merge_cells("A19:F19");
summary.merge_cells("A20:F20")
summary.get_range("A17:F17").format = header_fmt
summary.get_range("A1:F20").format.wrap_text = True
summary.freeze_panes.freeze_rows(2)

Character sheet
char_headers = ["#","ID","Name","Slug","Department","Code","Agent
Function","Rarity","Tier","Source","Origin","Status","Confidence","Photo
Page","Power","Precision","Speed","Resilience","Intelligence","Teamwork","Synced
Agents","Workflow Summary","Proof Gate"]
char_rows = []
for c in characters:
char_rows.append([
c["number"],c["id"],c["name"],c["slug"],c["department"],c["department_code"],
c["agent_function"],c["rarity"],c["rarity_tier"],c["source"],c["record_origin"],c["status"],c["confide
nce"],
c["visual"].get("primary_photo_page") or "",c["stats"]["power"],c["stats"]["precision"],c["stats"]
["speed"],
c["stats"]["resilience"],c["stats"]["intelligence"],c["stats"]["teamwork"],
" | ".join(c["workflow"]["synced_agents"]),c["workflow"]["summary"],c["workflow"]["proof_gate"]
])
char_sheet.get_range_by_indexes(0,0,len(char_rows)+1,len(char_headers)).values =
[char_headers] + char_rows
char_sheet.get_range(f"A1:W1").format = header_fmt
char_sheet.freeze_panes.freeze_rows(1)
char_sheet.tables.add(f"A1:W{len(char_rows)+1}", True, "CharacterRosterTable")
char_sheet.get_range(f"A2:A{len(char_rows)+1}").format.number_format = "0"
char_sheet.get_range(f"O2:T{len(char_rows)+1}").conditional_formats.add_data_bar({"color":"#
D4AF37","gradient":True})
char_sheet.get_range(f"A1:W{len(char_rows)+1}").format.wrap_text = True

Registry sheet
reg_headers = ["#","Registry ID","Name","Record
Kind","Type","Version","Status","Artifact","Collection","Marketplace Language","Card
Language","Capabilities"]
reg_rows =
[[r["catalog_number"],r["registry_id"],r["name"],r["record_kind"],r["type"],r["version"],r["status"]
,r["artifact"],r["collection"],r["marketplace_language"],r["card_language"]," |
".join(r["capabilities"])] for r in registry]
reg_sheet.get_range_by_indexes(0,0,len(reg_rows)+1,len(reg_headers)).values =
[reg_headers]+reg_rows
reg_sheet.get_range("A1:L1").format = header_fmt
reg_sheet.freeze_panes.freeze_rows(1)
reg_sheet.tables.add(f"A1:L{len(reg_rows)+1}", True, "RegistryManifestTable")
reg_sheet.get_range(f"A1:L{len(reg_rows)+1}").format.wrap_text = True
```

```
Photo alignment
photo_headers = ["Page","Display Title","Record Kind","Matched Record ID","Canonical
Name","Status","Confidence","Occurrence","Total Duplicates","Thumbnail","Source File"]
photo_rows =
[[a["page"],a["display_title"],a["record_kind"],a["matched_record_id"],a["canonical_name"],a["ali
gnment_status"],a["confidence"],a["duplicate_occurrence"],a["duplicate_total"],a["thumbnail"],a[
"source_file"]] for a in alignment]
photo_sheet.get_range_by_indexes(0,0,len(photo_rows)+1,len(photo_headers)).values =
[photo_headers]+photo_rows
photo_sheet.get_range("A1:K1").format = header_fmt
photo_sheet.freeze_panes.freeze_rows(1)
photo_sheet.tables.add(f"A1:K{len(photo_rows)+1}", True, "PhotoAlignmentTable")
photo_sheet.get_range(f"A1:K{len(photo_rows)+1}").format.wrap_text = True

Taxonomy
tax_headers = ["Group","Key","Label","Value","Color"]
tax_values = [[r["taxonomy_group"],r["key"],r["label"],r["value"],r["color"]] for r in tax_rows]
tax_sheet.get_range_by_indexes(0,0,len(tax_values)+1,len(tax_headers)).values =
[tax_headers]+tax_values
tax_sheet.get_range("A1:E1").format = header_fmt
tax_sheet.freeze_panes.freeze_rows(1)
tax_sheet.tables.add(f"A1:E{len(tax_values)+1}", True, "TaxonomyTable")
tax_sheet.get_range(f"A1:E{len(tax_values)+1}").format.wrap_text = True

Widths (bounded, readable).
for s in [summary, char_sheet, reg_sheet, photo_sheet, tax_sheet]:
s.get_range("A:Z").format.autofit_columns()
s.get_range("A:Z").format.autofit_rows()

Cap text-heavy columns.
for rng, width in [
(char_sheet.get_range("B:B"), 24),(char_sheet.get_range("C:C"), 24),(char_sheet.get_range("E:E"),
24),
(char_sheet.get_range("G:G"), 30),(char_sheet.get_range("J:M"), 22),
(char_sheet.get_range("U:W"), 38),
(reg_sheet.get_range("B:C"), 28),(reg_sheet.get_range("H:L"), 36),
(photo_sheet.get_range("B:E"), 30),(photo_sheet.get_range("J:K"), 30),
(tax_sheet.get_range("C:D"), 38)
]:
rng.format.column_width = width

xlsx_path = OUT / "miseos_full_roster_taxonomy.xlsx"
SpreadsheetFile.export_xlsx(wb).save(xlsx_path)

Compact verification.
check = wb.inspect({
"kind": "table",
"range": "Summary!A1:F20",
```

```
"include": "values,formulas",
"table_max_rows": 20,
"table_max_cols": 6,
})
print(check.ndjson)
errors = wb.inspect({
"kind": "match",
"search_term": "#REF!|#DIV/0!|#VALUE!|#NAME?|#N/A",
"options": {"use_regex": True, "max_results": 50},
"summary": "final formula error scan",
})
print(errors.ndjson)

Manifest and checksums.
manifest = []
for path in sorted(OUT.rglob("*")):
if path.is_file():
digest = hashlib.sha256(path.read_bytes()).hexdigest()
manifest.append({"path": str(path.relative_to(OUT)), "bytes": path.stat().st_size, "sha256":
digest})
(OUT / "MANIFEST.json").write_text(json.dumps(manifest, indent=2) + "\n")

Zip package.
zip_path = ROOT / "miseos_full_roster_taxonomy.zip"
if zip_path.exists():
zip_path.unlink()
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:
for path in sorted(OUT.rglob("*")):
if path.is_file():
zf.write(path, arcname=f"miseos_full_roster_taxonomy/{path.relative_to(OUT)}")

print({
"zip": str(zip_path),
"xlsx": str(xlsx_path),
"characters": len(characters),
"registry": len(registry),
"ecosystem": len(characters)+len(registry),
"photo_pages": len(alignment),
"zip_bytes": zip_path.stat().st_size
})

{
"schema": "miseos.taxonomy.v2",
"official_model": {
"character_agents": 112,
"registry_manifests": 23,
"total_ecosystem_items": 135,
```

"canonical_statement": "112 character agents + 23 registry manifests = 135 MiseOS ecosystem items."
},
"record_kinds": {
"character_card": {
"count": 112,
"description": "Collectible brigade operator with mascot art, lore, workflow, stats, commands, and media extensions."
},
"registry_manifest": {
"count": 23,
"description": "Product infrastructure. Never counted as a character."
}
},
"departments": [
"Data & Insight Kitchen",
"Operations Kitchen",
"Pipeline Kitchen",
"Platform Kitchen",
"Reliability Kitchen",
"Security Kitchen"
],
"rarities": [
"Common",
"Epic",
"Genesis",
"Legendary",
"Mythic",
"Rare"
],
"registry_types": [
"agent",
"analytics",
"container",
"package",
"policy",
"schema",
"sdk",
"template",
"theme",
"workflow"
],
"media_slots": [
"image",
"reference_art",
"thumbnail",

"animation",
"video",
"model_3d",
"audio",
"ar"
],
"workflow_fields": [
"inputs",
"outputs",
"proof_gate",
"synced_agents",
"summary"
],
"stat_fields": [
"power",
"precision",
"

Yes — here's the secure way to wire this in.

`api/server.js` startup config block

Put this at the very top of `api/server.js`, before routes or module initialization, so the process exits immediately on bad config.

```
function requireEnv(name) {
  const value = process.env[name];
  if (!value || !value.trim()) {
    throw new Error(`Missing required environment variable: ${name}`);
  }
  return value.trim();
}

function optionalEnv(name, fallback = "") {
  const value = process.env[name];
  return value && value.trim() ? value.trim() : fallback;
}

function validateHexAddress(name, value) {
  if (!/^0x[a-fA-F0-9]{40}$/.test(value)) {
    throw new Error(`Invalid ${name}: expected a 0x-prefixed Ethereum address`);
  }
  return value.toLowerCase();
}

const config = Object.freeze({
  port: Number.parseInt(optionalEnv("PORT", "3000"), 10),
  manifestPublicKey: requireEnv("MANIFEST_PUBLIC_KEY"),
  rpcUrl: requireEnv("RPC_URL"),
  nftContractAddress: validateHexAddress(
    "NFT_CONTRACT_ADDRESS",
```

```
    requireEnv("NFT_CONTRACT_ADDRESS")
  ),
  frontendUrl: optionalEnv("FRONTEND_URL", "http://localhost:8080")
});

if (!Number.isInteger(config.port) || config.port < 1 || config.port > 65535) {
  throw new Error("Invalid PORT: expected an integer between 1 and 65535");
}
```

Then use `config.manifestPublicKey`, `config.rpcUrl`, and `config.nftContractAddress` everywhere else in the server.

Ed25519 verification in `api/server.js`

Integrate manifest verification like this:

```
import express from "express";
import helmet from "helmet";
import cors from "cors";
import { verifyManifest } from "./manifestVerifier.js";

const app = express();

app.use(helmet());
app.use(cors({ origin: config.frontendUrl, credentials: true }));
app.use(express.json({ limit: "1mb" }));

app.post("/api/manifest/verify", (req, res) => {
  const manifest = req.body;

  if (!manifest || typeof manifest !== "object") {
    return res.status(400).json({ valid: false, reason: "Missing manifest" });
  }

  const valid = verifyManifest(manifest, config.manifestPublicKey);
  if (!valid) {
    return res.status(400).json({ valid: false, reason: "Invalid signature" });
  }

  res.json({
    valid: true,
    manifestId: manifest.asset?.sha256?.slice(0, 16) || "unknown",
    policies: manifest.policies || {}
  });
});
```

That keeps signature validation server-side and fail-closed. The manifest verifier should canonicalize the unsigned payload, verify the Ed25519 signature, and reject any malformed or tampered manifest.

Allowlist registry security logic

The workflow registry should act as a strict execution allowlist, not a dynamic config store.

Core logic

- `workflow/registry.json` maps an `actionId` to a local module path.
- The executor only accepts action IDs that exist in the registry.
- The executor resolves modules from a fixed local directory only.
- Any unknown action, malformed path, or path traversal attempt is rejected.
- The API never accepts raw code, URLs, or user-supplied module paths.

Secure flow

1. User clicks a hotspot.
2. Frontend sends `actionId` to the API.
3. API checks the session, manifest policy, and NFT requirements.
4. API confirms `actionId` exists in `registry.json`.
5. API imports only the matching local workflow module.
6. API executes that module with a bounded timeout.
7. API returns only structured output.

Why this matters

This prevents:

- remote code injection,
- registry poisoning,
- path traversal,
- and hidden runtime execution paths.

A good rule is: if it is not in the registry, it does not run.

Manifest canonicalization and signing tests

Add unit tests that cover both canonicalization and signature flow.

`test/manifest.test.js`

```
import test from "node:test";
import assert from "node:assert/strict";
import fs from "node:fs";
import crypto from "node:crypto";

function canonicalize(value) {
  if (Array.isArray(value)) {
```

```
    return `[${value.map(canonicalize).join(",")}]`;
  }
  if (value && typeof value === "object") {
    return `{${Object.keys(value)
      .sort()
      .map((k) => JSON.stringify(k) + ":" + canonicalize(value[k]))
      .join(",")}}`;
  }
  return JSON.stringify(value);
}

test("canonicalization is stable", () => {
  const a = { b: 2, a: 1, z: { y: 9, x: 8 } };
  const b = { z: { x: 8, y: 9 }, a: 1, b: 2 };
  assert.equal(canonicalize(a), canonicalize(b));
});

test("sign and verify manifest payload", () => {
  const { privateKey, publicKey } = crypto.generateKeyPairSync("ed25519");
  const manifest = {
    version: "1.0.0",
    asset: { pngPath: "/assets/character.png", sha256: "a".repeat(64) },
    hotspots: [],
    policies: { clickConsent: "required", noEmbeddedCode: true, allowlistOnly: true
  };

  const unsigned = canonicalize(manifest);
  const sig = crypto.sign(null, Buffer.from(unsigned), privateKey);
  const ok = crypto.verify(null, Buffer.from(unsigned), publicKey, sig);

  assert.equal(ok, true);
});

test("tampered manifest fails verification", () => {
  const { privateKey, publicKey } = crypto.generateKeyPairSync("ed25519");
  const manifest = {
    version: "1.0.0",
    asset: { pngPath: "/assets/character.png", sha256: "a".repeat(64) },
    hotspots: [],
    policies: { clickConsent: "required", noEmbeddedCode: true, allowlistOnly: true
  };

  const unsigned = canonicalize(manifest);
  const sig = crypto.sign(null, Buffer.from(unsigned), privateKey);

  manifest.policies.allowlistOnly = false;
  const tampered = canonicalize(manifest);

  const ok = crypto.verify(null, Buffer.from(tampered), publicKey, sig);
  assert.equal(ok, false);
});
```

Practical hardening notes

A few details matter for production:

- Validate environment variables before creating the Express app.
- Fail immediately if any required secret or URL is missing.
- Keep the manifest public key in a secret manager or deployment secret.
- Keep `workflow/registry.json` immutable in runtime containers.
- Add a test that asserts every registry action points to a local module path.
- Keep CSP strict so the browser never becomes a code-loading surface.

`api/server.js` integration order

The safest startup order is:

1. validate environment,
2. initialize config,
3. set security middleware,
4. load manifest verifier and workflow executor,
5. register routes,
6. start the listener.

That order avoids half-initialized services and prevents requests from being served with missing trust roots.

About the extra Python text

The Python snippet you pasted looks like a separate asset-upload and roster-generation workflow, not part of this Node repo. It should stay isolated from the manifest/signing service so the two systems do not share trust boundaries.

Practical hardening notes

A few details matter for production:
• Validate environment variables before creating the Express app.
• Fail immediately if any required secret or URL is missing.
• Keep the manifest public key in a secret manager or deployment secret.
• Keep workflow/registry.json immutable in runtime containers.
• Add a test that asserts every registry action points to a local module path.
• Keep CSP strict so the browser never becomes a code-loading surface.
api/server.js integration order
The safest startup order is:

1. validate environment,
2. initialize config,

3. set security middleware,
4. load manifest verifier and workflow executor,
5. register routes,
6. start the listener.
   That order avoids half-initialized services and prevents requests from being served with missing trust roots.
   About the extra Python text
   The Python snippet you pasted looks like a separate asset-upload and roster-generation workflow, not part of this Node repo. It should stay isolated from the manifest/signing service so the two systems do not share trust boundaries.

Yes — the practical production posture is exactly that: validate config first, fail fast, keep trust roots and allowlists out of mutable runtime paths, and enforce a strict CSP so the browser cannot become a script-loading channel. [193] [194] [195] [196]

Production ordering

The startup order you listed is the right one:

1. Validate environment.
2. Initialize config.
3. Set security middleware.
4. Load manifest verifier and workflow executor.
5. Register routes.
6. Start the listener.

That sequence prevents partially initialized state from serving requests and keeps required trust material present before the app accepts traffic. [196] [193]

Registry immutability

For `workflow/registry.json`, the secure production rule is: bake it into the image or deploy it as a read-only artifact, and do not mount it from a writable path. The container should treat the registry as curated deployment content, not mutable app state, which aligns with immutable artifact and curated registry practices. [197] [198] [199]

A good extra control is a startup checksum check:

- compute a SHA-256 of `workflow/registry.json`,
- compare it against an expected digest from the image build or release manifest,
- and fail startup if they differ.

That makes runtime tampering obvious.

Registry path safety

Also keep the executor strict:

- only local relative module paths,
- no `..` segments,
- no absolute paths,
- no URLs,
- no environment-expanded paths.

That prevents registry poisoning from becoming code execution. The registry should be an allowlist of fixed workflow modules, not an editable router. [200] [201] [202]

CSP posture

For the frontend, use a strict CSP and keep it minimal:

- `default-src 'self'`
- `script-src 'self'` or nonce-based scripts
- `object-src 'none'`
- `base-uri 'none'`
- `frame-ancestors 'none'`

Modern strict CSP guidance favors nonce or hash-based script authorization to block untrusted script execution and reduce XSS risk. [203] [194] [195]

Startup sequencing in code

In practice, structure `api/server.js` like this:

```
validateEnv();
const config = loadConfig();
const app = createApp(config);
registerMiddleware(app, config);
registerRoutes(app, config);
app.listen(config.port);
```

That keeps the initialization lifecycle explicit and testable.

Python text isolation

The Python asset-upload / roster-generation snippet should stay in a separate repository or a separate tooling boundary. It is a different trust domain, uses different secrets, and should not share runtime assumptions with the Node manifest/signing service. Keeping those systems separate reduces the chance that upload credentials, roster generation, or bucket sync logic accidentally expand the attack surface of the clickable-character app. [198] [199]

Best test to add

Add one runtime test that fails if the workflow registry is mutable or points outside the repo:

- load `workflow/registry.json`,
- verify every path matches `^\\./[A-Za-z0-9._/-]+\\.js$`,
- reject any path containing `..`,
- reject any absolute path,
- reject empty modules,
- and verify the resolved file exists under `workflow/`.

That gives you a simple and very effective guardrail.

Would you like the exact `api/server.js` refactor now, with `validateEnv()`, `createApp()`, and the registry immutability check already wired in?

⁂

Add one runtime test that fails if the workflow registry is mutable or points outside the repo:

• load workflow/registry.json,
• verify every path matches ^\./[A-Za-z0-9._/-]+\.js$,
• reject any path containing ..,
• reject any absolute path,
• reject empty modules,
• and verify the resolved file exists under workflow/.
That gives you a simple and very effective guardrail.
Would you like the exact api/server.js refactor now, with validateEnv(), createApp(), and the registry immutability check already wired in?

Yes, please provide the refactored server.js with those checks
Actually, could you show how to implement the SHA-256 startup check
No thanks, how do I configure the strict CSP in the
express app

Use Helmet's CSP middleware with a per-request nonce in `res.locals`, then bind that nonce into any inline script/style tags you render. The safest default is `default-src 'self'`, `object-src 'none'`, `base-uri 'none'`, `frame-ancestors 'none'`, and a nonce-based `script-src`/`style-src` policy when inline content is unavoidable.[213] [214] [215] [216]

Express CSP setup

This is the clean pattern for your Express app:

```
import crypto from "crypto";
import helmet from "helmet";

function cspNonce(req, res, next) {
  res.locals.cspNonce = crypto.randomBytes(32).toString("hex");
  next();
}

function buildCsp(isDevelopment) {
  return helmet.contentSecurityPolicy({
    useDefaults: false,
    directives: {
      defaultSrc: ["'self'"],
      baseUri: ["'none'"],
      objectSrc: ["'none'"],
      frameAncestors: ["'none'"],
      imgSrc: ["'self'", "data:"],
      connectSrc: ["'self'"],
      scriptSrc: [
        "'self'",
        (req, res) => `'nonce-${res.locals.cspNonce}'`
      ],
      styleSrc: [
        "'self'",
        (req, res) => `'nonce-${res.locals.cspNonce}'`
      ],
      formAction: ["'self'"],
      upgradeInsecureRequests: isDevelopment ? null : []
    }
  });
}
```

Then register it before your routes:

```
app.use(cspNonce);
app.use(buildCsp(app.get("env") === "development"));
```

That follows Helmet's nonce pattern and lets you avoid `unsafe-inline` while still supporting controlled inline tags when needed. [214] [213]

HTML side

If you render inline script or style tags, include the nonce attribute on the tag:

```
<script nonce="REPLACE_WITH_RES_LOCALS_CSP_NONCE"></script>
```

For your repo, the simplest route is to avoid inline scripts entirely and keep `frontend/index.html` as a shell that loads `/app.js` from the same origin. That makes the CSP easier to keep strict and removes most nonce requirements. [215] [216] [217]

Development exception

In development, you may want to disable `upgrade-insecure-requests` because Safari can rewrite `http://localhost` to HTTPS and break local testing. Helmet supports setting that directive to `null` in development. [214]

Recommended CSP for this project

A practical policy for your clickable-image app is:

```
default-src 'self';
base-uri 'none';
object-src 'none';
frame-ancestors 'none';
img-src 'self' data:;
connect-src 'self' http://localhost:3000;
script-src 'self';
style-src 'self';
form-action 'self'
```

If you later add inline tags, switch `script-src` and `style-src` to nonce-based directives instead of widening the host allowlist. [216] [213] [215]

Why this is the right fit

Your app's browser surface should only load the app bundle, the canvas image, and API calls to the backend. It should not permit arbitrary script origins, plugin objects, or framing, because those are the common paths for turning a visual asset app into a code-loading surface. [217] [218] [215]

I can next give you the exact `api/server.js` refactor with:

- `validateEnv()`
- `createApp()`
- `sha256` startup check for `workflow/registry.json`
- the CSP middleware wired in
- and the manifest verifier + workflow registry checks in place.

⁂

next give you the exact api/server.js refactor with:

• validateEnv()
• createApp()
• sha256 startup check for workflow/registry.json
• the CSP middleware wired in
• and the manifest verifier + workflow registry checks in place. Implement a post-deployment integrity check for registry.json
Best practices for atomic updates to workflow registry files
Automating integrity checks for JSON configuration in CI/CD

Here's the exact refactor pattern you asked for: fail fast on environment, compute a SHA-256 integrity check for `workflow/registry.json` at startup, wire in strict CSP, and keep manifest verification + workflow resolution behind explicit checks. This is the right shape for a service whose runtime trust depends on static config and allowlisted modules. [233] [234] [235] [236]

`api/server.js`

```
import fs from "fs";
import path from "path";
import crypto from "crypto";
import express from "express";
import cors from "cors";
import helmet from "helmet";
import { fileURLToPath } from "url";
import { verifyManifest } from "./manifestVerifier.js";
import workflows from "../workflow/registry.json" assert { type: "json" };
import { executeWorkflow } from "./workflowExecutor.js";
import { createNonce, verifyWalletLogin } from "./auth.js";
import { hasRequiredNft } from "./nft.js";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ROOT = path.resolve(__dirname, "..");
const REGISTRY_PATH = path.join(ROOT, "workflow", "registry.json");
const REGISTRY_SHA256_PATH = path.join(ROOT, "workflow", "registry.json.sha256");

function requireEnv(name) {
  const value = process.env[name];
  if (!value || !value.trim()) throw new Error(`Missing required environment variable
  return value.trim();
}

function optionalEnv(name, fallback = "") {
  const value = process.env[name];
  return value && value.trim() ? value.trim() : fallback;
}

function validateHexAddress(name, value) {
  if (!/^0x[a-fA-F0-9]{40}$/.test(value)) {
    throw new Error(`Invalid ${name}: expected a 0x-prefixed Ethereum address`);
  }
```

```
  return value.toLowerCase();
}

function validateEnv() {
  const port = Number.parseInt(optionalEnv("PORT", "3000"), 10);
  if (!Number.isInteger(port) || port < 1 || port > 65535) {
    throw new Error("Invalid PORT: expected an integer between 1 and 65535");
  }

  return Object.freeze({
    port,
    manifestPublicKey: requireEnv("MANIFEST_PUBLIC_KEY"),
    rpcUrl: requireEnv("RPC_URL"),
    nftContractAddress: validateHexAddress("NFT_CONTRACT_ADDRESS", requireEnv("NFT_C
    frontendUrl: optionalEnv("FRONTEND_URL", "http://localhost:8080"),
    registrySha256: optionalEnv("REGISTRY_SHA256", "")
  });
}

function sha256File(filePath) {
  const data = fs.readFileSync(filePath);
  return crypto.createHash("sha256").update(data).digest("hex");
}

function startupIntegrityCheck(config) {
  if (!fs.existsSync(REGISTRY_PATH)) {
    throw new Error("workflow/registry.json is missing");
  }

  const actual = sha256File(REGISTRY_PATH);
  const expected =
    config.registrySha256 ||
    (fs.existsSync(REGISTRY_SHA256_PATH) ? fs.readFileSync(REGISTRY_SHA256_PATH, "ut

  if (!expected) {
    throw new Error("Missing workflow registry SHA-256 expected value");
  }

  if (actual !== expected) {
    throw new Error(`workflow/registry.json integrity check failed: expected ${expec
  }
}

function createApp(config) {
  const app = express();

  app.disable("x-powered-by");

  app.use((req, res, next) => {
    res.locals.cspNonce = crypto.randomBytes(32).toString("hex");
    next();
  });

  app.use(
    helmet({
      contentSecurityPolicy: false
```

```
  })
);

app.use(
  helmet.contentSecurityPolicy({
    useDefaults: false,
    directives: {
      defaultSrc: ["'self'"],
      baseUri: ["'none'"],
      objectSrc: ["'none'"],
      frameAncestors: ["'none'"],
      imgSrc: ["'self'", "data:"],
      connectSrc: ["'self'", config.frontendUrl, config.rpcUrl],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'"],
      formAction: ["'self'"],
      upgradeInsecureRequests: []
    }
  })
);

app.use(cors({ origin: config.frontendUrl, credentials: true }));
app.use(express.json({ limit: "1mb" }));

app.get("/health", (req, res) => {
  res.json({ status: "ok", uptime: process.uptime() });
});

app.post("/api/manifest/verify", (req, res) => {
  const manifest = req.body;
  if (!manifest || typeof manifest !== "object") {
    return res.status(400).json({ valid: false, reason: "Missing manifest" });
  }
  const valid = verifyManifest(manifest, config.manifestPublicKey);
  if (!valid) return res.status(400).json({ valid: false, reason: "Invalid signatu
  return res.json({
    valid: true,
    manifestId: manifest.asset?.sha256?.slice(0, 16) || "unknown",
    policies: manifest.policies || {}
  });
});

app.post("/api/auth/nonce", (req, res) => {
  const { address } = req.body || {};
  if (!address || !/^0x[a-fA-F0-9]{40}$/.test(address)) {
    return res.status(400).json({ error: "Invalid Ethereum address" });
  }
  const nonce = createNonce(address);
  res.json({ nonce, address });
});

app.post("/api/auth/verify", async (req, res) => {
  const { address, message, signature } = req.body || {};
  if (!address || !message || !signature) {
    return res.status(400).json({ error: "Missing required fields" });
  }
```

```
    const ok = verifyWalletLogin(address, message, signature);
    if (!ok) return res.status(401).json({ error: "Invalid signature" });

    const nftAccess = await hasRequiredNft({
      rpcUrl: config.rpcUrl,
      contractAddress: config.nftContractAddress,
      ownerAddress: address
    });

    res.json({
      token: crypto.randomBytes(32).toString("hex"),
      address: address.toLowerCase(),
      nftAccess
    });
  });

  app.post("/api/authorize-action", (req, res) => {
    const { actionId } = req.body || {};
    if (!actionId || !Object.hasOwn(workflows, actionId)) {
      return res.json({ allowed: false, reason: "Action not in allowlist" });
    }
    res.json({
      allowed: true,
      context: {
        actionId,
        authorized: true,
        timestamp: Date.now()
      }
    });
  });

  app.post("/api/execute-workflow", async (req, res) => {
    const { actionId, context, params } = req.body || {};
    if (!actionId || !context || !Object.hasOwn(workflows, actionId)) {
      return res.status(403).json({ error: "Unauthorized workflow execution" });
    }

    try {
      const result = await executeWorkflow(actionId, { ...context, params });
      res.json({ ok: true, actionId, result, executedAt: Date.now() });
    } catch (err) {
      res.status(500).json({ error: "Workflow execution failed", details: err.message
    }
  });

  return app;
}

const config = validateEnv();
startupIntegrityCheck(config);

const app = createApp(config);
app.listen(config.port, () => {
  console.log(`API listening on ${config.port}`);
});
```

Post-deployment registry integrity check

For a runtime integrity check after deployment, make the container emit the expected registry hash during build, then verify it on boot and on a health endpoint. A simple pattern is:

- store `REGISTRY_SHA256` in the deployment environment,
- compare it against the current file hash at boot,
- expose `/health` so orchestration can detect tampering or drift.

That is the same basic logic used in file-integrity verification workflows: compare a computed SHA-256 to a trusted reference and fail if they differ. [234] [233]

Atomic updates for `workflow/registry.json`

Best practice is to update the registry atomically, not in place:

- write a new file, for example `registry.json.new`,
- fsync it,
- rename it over the old file in one atomic move,
- update the paired hash file at the same time,
- and deploy both together in one release artifact.

That avoids partial reads during updates and makes the registry/hash pair consistent across rollouts. In containers, the better option is even simpler: do not mutate it at runtime at all; replace the whole image or mounted config version. [237] [238]

CI/CD integrity automation

In CI/CD, add a job that:

- computes `sha256` for `workflow/registry.json`,
- compares it to a tracked expected value,
- runs the runtime registry-path test,
- and blocks the merge if any check fails.

This is the same general integrity pattern used for artifact verification in pipelines: a trusted digest is compared against the generated file before deploy. [239] [240] [233]

CSP in Express

You already have the strict CSP approach; the key production rule is to keep it header-based and avoid inline scripts entirely when possible. For this app, the simplest safe setting is `script-src 'self'`, no `unsafe-inline`, no `unsafe-eval`, `object-src 'none'`, `base-uri 'none'`, and `frame-ancestors 'none'`. [236] [241] [242]

If you want, I can now provide the matching:

- `test/registry.test.js`,

- `test/manifest.test.js`,
- and a small `workflow/registry.json.sha256` generation script.

⁂

now provide the matching:

• test/registry.test.js,
• test/manifest.test.js,
• and a small workflow/registry.json.sha256 generation script. Please provide the test/registry.test.js and registry.test.js files
Can you generate the registry.json.sha256 script first
Actually, I'd like all three items mentioned in the next steps

Here are all three items together: a registry integrity test, a manifest signing test, and a tiny SHA-256 generation script for `workflow/registry.json`. The registry test checks the allowlist shape and ensures every entry resolves to a local file under `workflow/`, which is the key guardrail for preventing registry-driven code injection.[256] [257] [258]

`test/registry.test.js`

```
import test from "node:test";
import assert from "node:assert/strict";
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ROOT = path.resolve(__dirname, "..");
const WORKFLOW_DIR = path.join(ROOT, "workflow");
const REGISTRY_PATH = path.join(WORKFLOW_DIR, "registry.json");

function loadRegistry() {
  const raw = fs.readFileSync(REGISTRY_PATH, "utf8");
  return JSON.parse(raw);
}

function isSafeRelativeJsPath(p) {
  return /^\.\/[A-Za-z0-9._/-]+\.js$/.test(p) && !p.includes("..") && !path.isAbsolu
}

test("workflow registry is immutable-shaped and local-only", () => {
  const registry = loadRegistry();
  assert.equal(typeof registry, "object");
  assert.ok(registry && !Array.isArray(registry));

  const entries = Object.entries(registry);
  assert.ok(entries.length > 0);

  for (const [actionId, modulePath] of entries) {
```

```
    assert.equal(typeof actionId, "string");
    assert.equal(typeof modulePath, "string");
    assert.ok(actionId.trim().length > 0);
    assert.ok(modulePath.trim().length > 0);
    assert.ok(isSafeRelativeJsPath(modulePath), `Unsafe path for ${actionId}: ${modul

    const resolved = path.resolve(WORKFLOW_DIR, modulePath);
    assert.ok(resolved.startsWith(WORKFLOW_DIR + path.sep), `Escapes workflow dir: $
    assert.ok(fs.existsSync(resolved), `Missing workflow module: ${resolved}`);
    assert.ok(fs.statSync(resolved).isFile(), `Not a file: ${resolved}`);
  }
});

test("workflow registry contains no duplicate or empty modules", () => {
  const registry = loadRegistry();
  const values = Object.values(registry);

  for (const v of values) {
    assert.equal(typeof v, "string");
    assert.ok(v.trim().length > 0);
  }

  const unique = new Set(values);
  assert.equal(unique.size, values.length, "Duplicate module paths found");
});
```

test/manifest.test.js

```
import test from "node:test";
import assert from "node:assert/strict";
import crypto from "node:crypto";

function canonicalize(value) {
  if (Array.isArray(value)) return `[${value.map(canonicalize).join(",")}]`;
  if (value && typeof value === "object") {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ":" + canonical
  }
  return JSON.stringify(value);
}

function signManifest(manifest, privateKey) {
  const unsigned = { ...manifest };
  delete unsigned.signature;
  const payload = canonicalize(unsigned);
  return crypto.sign(null, Buffer.from(payload), privateKey).toString("base64");
}

function verifyManifest(manifest, publicKey) {
  const { signature, ...unsigned } = manifest;
  const payload = canonicalize(unsigned);
  return crypto.verify(null, Buffer.from(payload), publicKey, Buffer.from(signature.
}

test("canonicalization is stable across key order", () => {
  const a = { b: 2, a: 1, z: { y: 9, x: 8 } };
```

```
  const b = { z: { x: 8, y: 9 }, a: 1, b: 2 };
  assert.equal(canonicalize(a), canonicalize(b));
});

test("signing and verification succeed on the same manifest", () => {
  const { privateKey, publicKey } = crypto.generateKeyPairSync("ed25519");

  const manifest = {
    version: "1.0.0",
    asset: { pngPath: "/assets/character.png", sha256: "a".repeat(64) },
    hotspots: [
      { id: "h1", name: "hotspot", shape: { type: "rect", x: 0, y: 0, w: 10, h: 10 }
    ],
    policies: { clickConsent: "required", noEmbeddedCode: true, allowlistOnly: true
  };

  const signature = signManifest(manifest, privateKey);
  const signed = { ...manifest, signature: { alg: "ed25519", keyId: "dev-key-1", valu

  assert.equal(verifyManifest(signed, publicKey), true);
});

test("tampering invalidates the signature", () => {
  const { privateKey, publicKey } = crypto.generateKeyPairSync("ed25519");

  const manifest = {
    version: "1.0.0",
    asset: { pngPath: "/assets/character.png", sha256: "a".repeat(64) },
    hotspots: [],
    policies: { clickConsent: "required", noEmbeddedCode: true, allowlistOnly: true
  };

  const signature = signManifest(manifest, privateKey);
  const signed = { ...manifest, signature: { alg: "ed25519", keyId: "dev-key-1", valu

  signed.policies.allowlistOnly = false;
  assert.equal(verifyManifest(signed, publicKey), false);
});
```

`workflow/registry.json.sha256` generation script

Save this as `scripts/gen-registry-sha256.mjs`:

```
import fs from "fs";
import crypto from "crypto";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ROOT = path.resolve(__dirname, "..");
const REGISTRY_PATH = path.join(ROOT, "workflow", "registry.json");
const OUT_PATH = path.join(ROOT, "workflow", "registry.json.sha256");

const data = fs.readFileSync(REGISTRY_PATH);
```

```
const hash = crypto.createHash("sha256").update(data).digest("hex");

fs.writeFileSync(OUT_PATH, `${hash}  registry.json\n`);
console.log(`Wrote ${OUT_PATH}`);
```

Run it with:

```
node scripts/gen-registry-sha256.mjs
```

How these fit together

The registry test protects the allowlist boundary by ensuring the workflow map stays local and file-backed only. The manifest test protects the signed-manifest pipeline by checking both canonicalization and signature behavior. The hash-generation script gives you a repeatable integrity reference that your server can verify at startup or in CI/CD, which is a standard file-integrity pattern for deployment pipelines. [258] [259] [260]

If you want, I can next provide a small `npm test` setup snippet and a CI job that runs both tests plus the registry hash check automatically.

⁂

next provide a small npm test setup snippet and a Cl job that runs both tests plus the registry hash check automatically.

↑
Yes, please provide the npm test setup and CI pipeline configuration
No thanks, I'II set up the Cl testing flow myself tor now
Actually, can you explain how to sign the registry file instead

Yes — signing `workflow/registry.json` is a good idea if you want an explicit integrity artifact for deployment. The safest pattern is: canonicalize the registry JSON, sign it with your Ed25519 dev/private key, store the public key separately, and have startup or CI verify the signature before accepting the file. [274] [275] [276]

How to sign it

Use a separate signature file, for example:

- `workflow/registry.json`
- `workflow/registry.json.sig`
- `workflow/registry.json.pub` or a deployment-stored public key

That keeps the registry itself human-readable and lets your app verify both the hash and the signature at boot.

scripts/sign-registry.mjs

```
import fs from "fs";
import crypto from "crypto";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ROOT = path.resolve(__dirname, "..");

const REGISTRY_PATH = path.join(ROOT, "workflow", "registry.json");
const SIG_PATH = path.join(ROOT, "workflow", "registry.json.sig");
const PRIVATE_KEY_PATH = process.argv[^18_2];

if (!PRIVATE_KEY_PATH) {
  console.error("Usage: node scripts/sign-registry.mjs <private-key-pem>");
  process.exit(1);
}

function canonicalize(value) {
  if (Array.isArray(value)) return `[${value.map(canonicalize).join(",")}]`;
  if (value && typeof value === "object") {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ":" + canonical
  }
  return JSON.stringify(value);
}

const raw = fs.readFileSync(REGISTRY_PATH, "utf8");
const parsed = JSON.parse(raw);
const payload = Buffer.from(canonicalize(parsed));
const privateKeyPem = fs.readFileSync(PRIVATE_KEY_PATH, "utf8");

const sig = crypto.sign(null, payload, privateKeyPem).toString("base64");
fs.writeFileSync(SIG_PATH, sig + "\n");
console.log(`Wrote ${SIG_PATH}`);
```

scripts/verify-registry.mjs

```
import fs from "fs";
import crypto from "crypto";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ROOT = path.resolve(__dirname, "..");

const REGISTRY_PATH = path.join(ROOT, "workflow", "registry.json");
const SIG_PATH = path.join(ROOT, "workflow", "registry.json.sig");
const PUBLIC_KEY_PATH = process.argv[^18_2];

if (!PUBLIC_KEY_PATH) {
  console.error("Usage: node scripts/verify-registry.mjs <public-key-pem>");
```

```
  process.exit(1);
}

function canonicalize(value) {
  if (Array.isArray(value)) return `[${value.map(canonicalize).join(",")}]`;
  if (value && typeof value === "object") {
    return `{${Object.keys(value).sort().map(k => JSON.stringify(k) + ":" + canonical
  }
  return JSON.stringify(value);
}

const raw = fs.readFileSync(REGISTRY_PATH, "utf8");
const parsed = JSON.parse(raw);
const payload = Buffer.from(canonicalize(parsed));
const sig = Buffer.from(fs.readFileSync(SIG_PATH, "utf8").trim(), "base64");
const publicKeyPem = fs.readFileSync(PUBLIC_KEY_PATH, "utf8");

const ok = crypto.verify(null, payload, publicKeyPem, sig);
if (!ok) {
  throw new Error("Registry signature verification failed");
}
console.log("Registry signature OK");
```

Why use signature plus hash

A SHA-256 file gives you a quick tamper check, while an Ed25519 signature gives you authenticity as well. In practice:

- SHA-256 tells you the file is unchanged.
- Ed25519 tells you the file was produced by someone holding the private key.

That combination is stronger than hash-only integrity for deployment trust.

`package.json` test setup snippet

For Node's built-in test runner, this is enough:

```
{
  "scripts": {
    "test": "node --test"
  }
}
```

If you want all tests plus registry verification in one command:

```
{
  "scripts": {
    "test": "node --test test/*.test.js && node scripts/verify-registry.mjs secrets/
  }
}
```

Practical recommendation

For production, prefer:

- sign `workflow/registry.json` in CI,
- verify the signature at boot,
- verify the SHA-256 too if you want a quick secondary guard,
- and never let runtime code rewrite the registry or its signature files.

That way the allowlist remains a trusted deployment artifact instead of mutable app state.[277] [278] [279]

If you want, I can next give you the matching `test/registry-signature.test.js` and a tiny `workflow/registry.json.sig` verification hook for `api/server.js`.

⁂

next give you the matching test/registry-signature.test.js and a tiny workflow/registry.json.sig verification hook for api/server.js . Yes, please provide the signature test and the verification hook

I would like to see how to integrate this into a production CI flow
Can we stick to just hashing instead of digital signatures instead

Yes — if you want to stick to hashing only, that's perfectly reasonable for a mutable config artifact like `workflow/registry.json`, as long as you treat the expected hash as a trusted deployment input and keep the registry file immutable in the runtime container.[289] [290]

Hash-only verification hook

Add this small hook in `api/server.js` or a separate module:

```
import fs from "fs";
import crypto from "crypto";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ROOT = path.resolve(__dirname, "..");
const REGISTRY_PATH = path.join(ROOT, "workflow", "registry.json");
const REGISTRY_SHA256_PATH = path.join(ROOT, "workflow", "registry.json.sha256");

function sha256File(filePath) {
  return crypto.createHash("sha256").update(fs.readFileSync(filePath)).digest("hex")
}

function verifyRegistryHashOrThrow(expectedHash) {
```

```
  const actual = sha256File(REGISTRY_PATH);
  if (!expectedHash) {
    throw new Error("Missing expected registry SHA-256");
  }
  if (actual !== expectedHash.trim().split(/\s+/)[^19_0]) {
    throw new Error(`registry hash mismatch: expected ${expectedHash}, got ${actual}
  }
}
```

Startup integration

Use it during boot, right after env validation and before route registration:

```
const config = validateEnv();
const expectedRegistryHash =
  config.registrySha256 ||
  fs.readFileSync(REGISTRY_SHA256_PATH, "utf8").trim();

verifyRegistryHashOrThrow(expectedRegistryHash);
const app = createApp(config);
```

`test/registry-signature.test.js` replaced with hash test

If you are dropping signatures entirely, make the runtime test verify the digest file instead of signatures:

```
import test from "node:test";
import assert from "node:assert/strict";
import fs from "node:fs";
import crypto from "node:crypto";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ROOT = path.resolve(__dirname, "..");
const REGISTRY_PATH = path.join(ROOT, "workflow", "registry.json");
const HASH_PATH = path.join(ROOT, "workflow", "registry.json.sha256");

function sha256File(filePath) {
  return crypto.createHash("sha256").update(fs.readFileSync(filePath)).digest("hex")
}

test("workflow registry hash matches digest file", () => {
  const actual = sha256File(REGISTRY_PATH);
  const expected = fs.readFileSync(HASH_PATH, "utf8").trim().split(/\s+/)[^19_0];
  assert.equal(actual, expected);
});
```

CI flow for hashing

A clean CI flow looks like this:

1. Build or checkout the repo.
2. Run `node scripts/gen-registry-sha256.mjs`.
3. Fail if `workflow/registry.json.sha256` changed unexpectedly.
4. Run `npm test`.
5. Deploy only if both the test and hash check pass.

That gives you integrity verification without introducing key management for signatures. Hash-based integrity is common in pipelines and works well when the trusted digest is supplied by the build or release process. [291] [290]

When hashing is enough

Hash-only is enough if:

- the registry is generated in a trusted build step,
- the expected digest is stored in CI or deployment metadata,
- and the container runs the file as read-only.

If you need provenance or offline authenticity, signatures are stronger. But for an internal deployment artifact, SHA-256 can be a simpler and perfectly solid control. [292] [290]

Practical recommendation

If you choose hashing only, keep these in place:

- immutable runtime file,
- startup digest check,
- CI digest generation,
- and registry path validation.

That way the allowlist stays both local and tamper-evident.

I can next give you the exact `api/server.js` hash-only refactor with the registry digest check wired in and the CSP middleware left intact.

⁂

1. https://www.cs.toronto.edu/~arnold/427/18s/427_18S/tutorials/stegosploit/StegosploitTut.pdf
2. https://www.reddit.com/r/programming/comments/2lygcb/serve_javascript_within_picture_files/
3. https://github.com/acuciureanu/png-payload-injector
4. https://expobrain.net/2014/09/26/hide-javascript-code-into-images/
5. https://github.com/god233012yamil/ExeSteganography
6. https://github.com/hugohrban/steganography/blob/main/README.md

7. https://github.com/CleasbyCode/pdvzip
8. http://www.cs.toronto.edu/~arnold/427/20s/427_20S/stego/presentation.pdf
9. https://medium.com/@abh200731/room-105-steganography-85f6ceadc53d
10. https://cryptomex.org/Publicaciones/i2cs06.pdf
11. https://www.cs.toronto.edu/~arnold/427/18s/427_18S/tutorials/stegosploit/StegosploitTut.pdf
12. https://github.com/acuciureanu/png-payload-injector
13. https://expobrain.net/2014/09/26/hide-javascript-code-into-images/
14. miseos-ecosystem-135.json
15. photo-alignment.csv
16. https://www.w3.org/TR/PNG-Misc.html
17. https://stackoverflow.com/questions/13883064/securityerror-load-an-image-in-canvas-modify-it-and-create-dataurl
18. https://www.web3.university/article/how-to-create-an-nft-on-ethereum-tutorial
19. https://aws.amazon.com/blogs/database/build-nft-metadata-access-control-with-ethereum-signatures-and-aws-lambda-authorizers/
20. https://www.semanticscholar.org/paper/c194887be58a84d9ca74f53513ba270e62650eee
21. https://www.semanticscholar.org/paper/4e501f09123c54a99d7470d77522188833b4a046
22. https://www.semanticscholar.org/paper/ae1a38c8b9056ad953ed9fe977e35f6bff2d2eeb
23. https://securityaffairs.com/37302/hacking/stegosploit-malware-images.html
24. https://www.uptycs.com/blog/threat-research-report-team/denial-of-servicedos-and-arbitrary-file-read-vulnerability-in-imagemagick
25. https://gist.github.com/GeoffMahugu/69a231022198d19e7bead651860a7bbf
26. https://docs.near.org/tutorials/nfts/js/minting
27. https://www.youtube.com/watch?v=9oERTH9Bkw0
28. https://www.exploit-db.com/exploits/37699
29. https://ieeexplore.ieee.org/document/11166796/
30. https://www.semanticscholar.org/paper/0f20963d0d3641290f924a89177fcd92afb765c3
31. https://www.semanticscholar.org/paper/e03d29495d2d79b5939bdb2f8bf3487e8b53dd6e
32. https://www.semanticscholar.org/paper/24e3022fb8b07590a8f384247ece85d27d37b237
33. https://www.semanticscholar.org/paper/83da982142bc2d7a8f851a7e895b2e7d2bdeb2e1
34. https://www.w3.org/TR/CSP3/
35. https://web.dev/articles/sandboxed-iframes
36. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
37. https://developer.chrome.com/docs/apps/manifest/sandbox
38. https://web.dev/articles/strict-csp
39. https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/12-Test_for_Content_Security_Policy
40. https://www.semanticscholar.org/paper/92ab2fafa70fed9a9217d4672d49f3cee116d24d
41. https://www.osti.gov/servlets/purl/1150849/

42. https://www.taylorfrancis.com/books/9781439812846
43. https://www.semanticscholar.org/paper/6843b31fced82460fca215563c11c67430b5f072
44. http://link.springer.com/10.1007/11802372
45. https://www.cs.columbia.edu/~suman/6183_slides/workers-sandbox-csp.pdf
46. https://medium.com/@mohammedaashik.f2022/a-comprehensive-analysis-of-content-security-policy-errors-involving-eval-and-strategies-for-381f97e6cd24
47. https://nfts.co.uk/sites/default/files/policies/IT_Access_Control_Policy.pdf
48. https://developer.chrome.com/docs/extensions/how-to/security/sandboxing-eval?hl=th
49. https://ieeexplore.ieee.org/document/8967424/
50. https://ieeexplore.ieee.org/document/8783521/
51. https://www.semanticscholar.org/paper/2b2269932e5d5a94e8668b9fd25876c5da9ea7dc
52. https://www.w3.org/TR/CSP3/
53. https://web.dev/articles/sandboxed-iframes
54. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
55. https://aws.amazon.com/blogs/database/build-nft-metadata-access-control-with-ethereum-signatures-and-aws-lambda-authorizers/
56. https://ieeexplore.ieee.org/document/8967424/
57. https://web.dev/articles/strict-csp
58. https://www.w3.org/TR/CSP3/
59. https://web.dev/articles/sandboxed-iframes
60. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
61. https://mermaidpreview.com/blog/architecture-documentation-best-practices.html
62. https://mermaid.ai/open-source/syntax/architecture.html
63. https://web.dev/articles/strict-csp
64. https://www.mdpi.com/2410-387X/7/2/21/pdf?version=1681435731
65. https://mermaideditor.lol/blog/mermaid-architecture-diagram-guide
66. https://revision.app/blog/mermaid-architecture-diagram
67. https://www.mermaidcreator.com/blog/mermaid-block-diagram-system-architecture
68. https://onuml.com/blog/mermaid-flowchart-guide
69. https://www.mermaidcreator.com/blog/mermaid-large-diagram-optimization-performance
70. https://www.mermaidcreator.com/blog/system-architecture-diagrams-mermaid
71. https://www.youtube.com/watch?v=aMMDMQvSzmc
72. https://github.com/digitalbazaar/ed25519-signature-2018/blob/main/package.json
73. https://dl.acm.org/doi/pdf/10.1145/3576915.3623200
74. https://zenodo.org/record/5091523/files/Edge2BC a Practical Approach for Edge-to-Blockchain IoT Transactions.pdf
75. https://jurnal.ciptamediaharmoni.id/index.php/jsitik/article/download/309/160
76. https://csp.withgoogle.com/docs/strict-csp.html

77. https://www.w3.org/TR/CSP3/
78. https://web.dev/articles/sandboxed-iframes
79. https://docs.capisc.io/recipes/security/key-rotation/
80. https://dl.acm.org/doi/pdf/10.1145/3576915.3623200
81. https://github.com/w3c/webappsec-csp/issues/97
82. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
83. https://ijsra.net/node/3838
84. https://asmedigitalcollection.asme.org/IPC/proceedings/IPC2024/88575/V004T06A002/1210686
85. http://link.springer.com/10.1007/s00381-018-4023-3
86. https://journals.nipes.org/index.php/jstrissue/article/view/2663
87. https://www.semanticscholar.org/paper/356d08f289700cb47d8fe08d371aff5c74d70798
88. https://link.springer.com/10.1007/s00381-025-07093-9
89. https://www.w3.org/TR/2016/WD-CSP3-20160913/
90. https://www.aem.live/docs/csp
91. https://web.dev/articles/strict-csp?hl=hi
92. https://stackoverflow.com/questions/53836164/how-to-define-a-content-security-policy-csp-that-utilizes-strict-dynamic-but
93. https://gist.github.com/oscarychen/c840ac6c4635b70101a16767e8a569a5
94. https://csp.withgoogle.com/docs/faq.html
95. https://www.brandonchecketts.com/archives/ssh-ed25519-key-best-practices-for-2025
96. https://www.nature.com/articles/s41598-026-43667-6
97. https://www.mdpi.com/2227-9032/13/8/946
98. text-2.txt
99. miseos_character_master_inventory-2.csv
100. https://docs.capisc.io/recipes/security/key-rotation/
101. https://csp.withgoogle.com/docs/strict-csp.html
102. https://www.w3.org/TR/CSP3/
103. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
104. https://web.dev/articles/sandboxed-iframes
105. miseos_character_master_inventory.csv
106. https://www.w3.org/TR/CSP3/
107. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
108. https://csp.withgoogle.com/docs/strict-csp.html
109. https://www.frontendtechlead.com/learn-docker/docker-for-backend
110. https://dev.to/mspilari/dockerizing-an-expressjs-api-with-postgresql-database-for-testing-and-production-1pjj
111. https://dev.to/wdsega/dockerrong-qi-hua-bu-shu-nodejsying-yong-zui-jia-shi-jian-k9h
112. https://identity.foundation/JcsEd25519Signature2020/

113. https://docs.capisc.io/recipes/security/key-rotation/
114. http://arxiv.org/pdf/1902.08318.pdf
115. https://downloads.hindawi.com/journals/scn/2019/9629034.pdf
116. https://arxiv.org/pdf/2107.10164.pdf
117. https://matrix-org.github.io/matrix-rust-sdk/src/matrix_sdk_crypto/olm/utility.rs.html
118. https://blog.cloudsigma.com/how-to-deploy-a-node-js-express-js-app-with-docker-on-ubuntu-20-04/
119. https://go.dev/src/crypto/ed25519/ed25519vectors_test.go
120. https://stackoverflow.com/questions/76743558/verify-digital-signature-signed-using-ed25519
121. https://dev.to/eneaslari/the-complete-guide-to-dockerizing-a-nodejs-express-app-with-database-file-uploads-1nck
122. https://github.com/w3cj/express-api-starter-ts/blob/main/package.json
123. https://arxiv.org/pdf/2202.13953.pdf
124. https://www.w3.org/TR/CSP3/
125. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
126. https://csp.withgoogle.com/docs/strict-csp.html
127. https://arxiv.org/pdf/2407.02522.pdf
128. https://arxiv.org/pdf/2310.07847.pdf
129. https://arxiv.org/pdf/1902.09217.pdf
130. http://arxiv.org/pdf/2112.10165.pdf
131. https://arxiv.org/pdf/1903.02613.pdf
132. https://figshare.com/articles/conference_contribution/Jack-in-the-box_An_Empirical_Study_of_JavaScript_Bundling_on_the_Web_and_its_Security_Implications/25304470/1/files/44724412.pdf
133. https://downloads.hindawi.com/journals/scn/2019/9629034.pdf
134. https://docs.deno.com/api/node/crypto/~/generateKeyPair
135. https://nodejs.org/download/release/v21.5.0/docs/api/webcrypto.html
136. https://gist.github.com/dahlia/7a182572e01a327ec25417739d6960c0
137. https://arxiv.org/pdf/2202.13953.pdf
138. http://arxiv.org/pdf/2403.08334.pdf
139. https://github.com/digitalbazaar/ed25519-verification-key-2018
140. https://mojoauth.com/keypair-generation/generate-keypair-using-ed25519-with-mongoose/
141. https://gist.github.com/adrianhopebailie/b7d65c52c6e5f21edc1297b55757b8f6
142. https://mojoauth.com/keypair-generation/generate-keypair-using-ed25519-with-javascript
143. https://github.com/java-crypto/cross_platform_crypto/blob/main/docs/generate_ed25519_keypair.md
144. https://github.com/BENMFeng/Ed25519_DS
145. https://www.compilenrun.com/docs/framework/express/express-security/express-security-best-practices/
146. https://apeiris.ai/integration/verify/
147. https://docs.sill.so/verification/verify-a-signature/
148. https://docs.digital.gov.krd/devsecops/02-dockerfile-best-practices.html

149. https://gauravkaushal.tech/blog/dockerfile-best-practices/
150. https://arxiv.org/pdf/1902.09217.pdf
151. https://arxiv.org/pdf/1903.02613.pdf
152. http://arxiv.org/pdf/2112.10165.pdf
153. https://figshare.com/articles/conference_contribution/Jack-in-the-box_An_Empirical_Study_of_JavaScript_Bundling_on_the_Web_and_its_Security_Implications/25304470/1/files/44724412.pdf
154. https://instadevops.com/blog/docker-production-best-practices/
155. https://arxiv.org/pdf/2407.02522.pdf
156. https://arxiv.org/pdf/2310.07847.pdf
157. https://arxiv.org/pdf/2202.13953.pdf
158. https://csp.withgoogle.com/docs/strict-csp.html
159. https://docs.deno.com/api/node/crypto/~/generateKeyPair
160. https://nodejs.org/download/release/v21.5.0/docs/api/webcrypto.html
161. https://gist.github.com/dahlia/7a182572e01a327ec25417739d6960c0
162. http://arxiv.org/pdf/2410.15289.pdf
163. http://arxiv.org/pdf/2501.08550.pdf
164. http://arxiv.org/pdf/2405.20561.pdf
165. https://arxiv.org/pdf/1907.04262.pdf
166. http://arxiv.org/pdf/2408.04939.pdf
167. https://www.verticaledgeai.ai/sample-package/verifier_cli.py
168. https://docs.nexart.io/docs/independent-verification
169. https://www.kuaiyizhi.cn/en/courses/docker/08-docker-production-best-practices/
170. https://instagit.com/vercel/next.js/react-docker-best-practices-large-app/
171. https://certifieddata.io/ed25519-ai-certificates
172. https://apeiris.ai/integration/verify/
173. https://docs.sill.so/verification/verify-a-signature/
174. https://certifieddata.io/sandbox/manifest
175. https://dev.to/roshan_ican/validating-environment-variables-in-nodejs-with-zod-2epn
176. https://jsdev.space/howto/env-ts-zod/
177. https://www.youtube.com/watch?v=ecMx33_4KGc
178. https://gauravkaushal.tech/blog/dockerfile-best-practices/
179. https://instadevops.com/blog/docker-production-best-practices/
180. https://docs.digital.gov.krd/devsecops/02-dockerfile-best-practices.html
181. http://arxiv.org/pdf/2112.10165.pdf
182. https://arxiv.org/pdf/2503.14443.pdf
183. https://arxiv.org/pdf/2502.13681.pdf
184. https://arxiv.org/pdf/2503.02770.pdf

185. http://arxiv.org/pdf/2405.06832.pdf
186. http://arxiv.org/pdf/2412.18109.pdf
187. https://arxiv.org/pdf/2407.02522.pdf
188. https://arxiv.org/pdf/2502.09117.pdf
189. https://dev.to/rohan_mirjankar/introducing-envguard-zero-boilerplate-environment-variable-validation-for-nodejs-52mi
190. https://www.verticaledgeai.ai/sample-package/verifier_cli.py
191. https://dev.to/madeburo/type-safe-environment-validation-3mdi
192. https://guaracloud.com/en/blog/environment-variables-docker-production/
193. https://www.compilenrun.com/docs/framework/express/express-best-practices/express-configuration-management/
194. https://web.dev/articles/strict-csp
195. https://csp.withgoogle.com/docs/strict-csp.html
196. https://github.com/goldbergyoni/nodebestpractices
197. https://letsbuildsolutions.com/blog/devops/immutable-infrastructure-in-practice-golden-images-container-registries-and-deployment-patterns-that-eliminate-configuration-drift/
198. https://docs.cloudraya.com/panel/main/05-storage-services/cloudraya-container-registry/container-registry-best-practices
199. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux_atomic_host/7/html/container_security_guide/keeping_containers_fresh_and_updateable
200. https://arxiv.org/pdf/1902.09217.pdf
201. https://arxiv.org/pdf/1903.02613.pdf
202. https://figshare.com/articles/conference_contribution/Jack-in-the-box_An_Empirical_Study_of_JavaScript_Bundling_on_the_Web_and_its_Security_Implications/25304470/1/files/44724412.pdf
203. https://developer.chrome.com/docs/lighthouse/best-practices/csp-xss
204. https://link.springer.com/10.1007/s41237-022-00159-x
205. https://www.semanticscholar.org/paper/0b8cdaa31bf98ba421c4fbb5ff2618292b922f90
206. https://saemobilus.sae.org/papers/integrating-functional-non-functional-design-verification-embedded-software-systems-2009-01-0152
207. https://www.semanticscholar.org/paper/7469136ef0e11dfcc4c10a5f5cb7dc611956075b
208. https://arxiv.org/pdf/2501.15475.pdf
209. http://arxiv.org/pdf/2410.16720.pdf
210. http://arxiv.org/pdf/2401.08595.pdf
211. https://owasp.org/www-community/controls/Content_Security_Policy
212. https://dev.to/dortort/idempotent-dockerfiles-desirable-ideal-or-misplaced-objective-e7c
213. https://helmetjs.github.io/faq/csp-nonce-example/
214. https://github.com/helmetjs/helmet/blob/main/middlewares/content-security-policy/README.md
215. https://web.dev/articles/strict-csp
216. https://csp.withgoogle.com/docs/strict-csp.html

217. https://developer.chrome.com/docs/lighthouse/best-practices/csp-xss
218. https://owasp.org/www-community/controls/Content_Security_Policy
219. http://arxiv.org/pdf/2410.14924.pdf
220. http://arxiv.org/pdf/2209.01541.pdf
221. https://research.chalmers.se/publication/517679/file/517679_Fulltext.pdf
222. https://oneuptime.com/blog/post/2026-01-15-content-security-policy-csp-react/view
223. https://shattered.io/content-security-policy-nodejs/
224. https://stackpractices.com/guides/complete-guide-content-security-policy/
225. https://stackoverflow.com/questions/77765747/helmet-js-express-content-security-policy-errors-with-angular-ngcspnonce
226. https://stackoverflow.com/questions/77468604/is-setting-a-csp-nonce-supposed-to-work-magically-in-node-express
227. https://oneuptime.com/blog/post/2026-01-25-helmet-security-expressjs/view
228. https://borstch.com/blog/development/angular-and-content-security-policy-csp-best-practices
229. https://web.dev/articles/strict-csp?hl=pl
230. https://figshare.com/articles/conference_contribution/Honey_I_Cached_our_Security_Tokens_Re-usage_of_Security_Tokens_in_the_Wild/25550853/1/files/45469566.pdf
231. http://arxiv.org/pdf/2309.07782.pdf
232. https://arxiv.org/pdf/1611.02875.pdf
233. https://theproductguy.in/blogs/hash-verifier-ci/
234. https://dev.to/orkhanhuseyn/verifying-integrity-of-files-using-nodejs-1gnd
235. https://github.com/goldbergyoni/nodebestpractices
236. https://web.dev/articles/strict-csp
237. https://letsbuildsolutions.com/blog/devops/immutable-infrastructure-in-practice-golden-images-container-registries-and-deployment-patterns-that-eliminate-configuration-drift/
238. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux_atomic_host/7/html/container_security_guide/keeping_containers_fresh_and_updateable
239. https://jadapps.app/security-tools/guides/verify-npm-package-supply-chain-integrity
240. https://www.cybersrely.com/slsa-1-1-implementation-in-ci-cd/
241. https://csp.withgoogle.com/docs/strict-csp.html
242. https://owasp.org/www-community/controls/Content_Security_Policy
243. https://arxiv.org/pdf/2202.13953.pdf
244. https://downloads.hindawi.com/journals/scn/2022/9473246.pdf
245. http://arxiv.org/pdf/2402.03555.pdf
246. https://downloads.hindawi.com/journals/wcmc/2022/5721917.pdf
247. http://arxiv.org/pdf/2403.08334.pdf
248. https://pmc.ncbi.nlm.nih.gov/articles/PMC9269824/
249. https://arxiv.org/pdf/2407.02522.pdf
250. https://bidda.com/verify

251. https://kodachi.cloud/kp/docs-assets/binaries/bin-json/integrity-check_rust_binary.json
252. https://github.com/nodejs/node/issues/13635
253. https://github.com/CurveLTD/Integrity-Checker
254. https://github.com/gitlabhq/gitlabhq/blob/master/doc/api/lint.md
255. https://github.com/shahradelahi/node-checksum
256. https://stackoverflow.com/questions/75947454/is-there-a-way-to-generate-sha256-or-similar-in-nodejs-of-big-files
257. https://nodejs.org/api/crypto.html
258. https://theproductguy.in/blogs/hash-verifier-ci/
259. https://dev.to/orkhanhuseyn/verifying-integrity-of-files-using-nodejs-1gnd
260. https://www.cybersrely.com/slsa-1-1-implementation-in-ci-cd/
261. http://arxiv.org/pdf/2411.19358.pdf
262. https://github.com/digitalcredentials/sha256-universal/blob/main/package.json
263. https://www.w3schools.com/nodejs/ref_hash.asp
264. https://www.w3schools.com/nodejs/nodejs_crypto.asp
265. https://github.com/dchest/fast-sha256-js/blob/master/package.json
266. https://nodejs.org/download/release/v21.5.0/docs/api/crypto.json
267. https://gist.github.com/acdcjunior/3c57b2b2e8df413aa127a05f61d136a7
268. https://github.com/remarkablemark/remarkablemark.github.io/blob/master/_posts/2021/2021-08-29-javascript-generate-sha-256-hexadecimal-hash.md
269. https://stackoverflow.com/questions/27970431/using-sha-256-with-nodejs-crypto
270. https://arxiv.org/pdf/2202.13953.pdf
271. http://arxiv.org/pdf/2405.06832.pdf
272. https://arxiv.org/pdf/2104.07460.pdf
273. https://arxiv.org/pdf/2301.05097v1.pdf
274. https://www.shs-conferences.org/articles/shsconf/pdf/2021/30/shsconf_qqr2021_07004.pdf
275. https://stackoverflow.com/questions/66476780/sign-verify-json-using-ed25519-keys-with-bouncy-castle-java
276. https://www.w3schools.com/nodejs/ref_sign.asp
277. https://theproductguy.in/blogs/hash-verifier-ci/
278. https://www.cybersrely.com/slsa-1-1-implementation-in-ci-cd/
279. https://dev.to/orkhanhuseyn/verifying-integrity-of-files-using-nodejs-1gnd
280. https://blog.npmjs.org/post/127671403050/testing-and-deploying-with-ordered-npm-run-scripts.html
281. https://docs.npmjs.com/cli/v8/commands/npm-test/
282. https://nodejs.cn/npm/cli/v7/commands/npm-test/
283. https://forum.freecodecamp.org/t/how-to-npm-test-js/55482
284. https://www.w3schools.com/nodejs/nodejs_package_json.asp
285. https://www.npmjs.com/package/test
286. https://www.mdpi.com/1424-8220/24/16/5106/pdf?version=1722997984

287. https://www.npmjs.com/package/testing
288. https://stackoverflow.com/questions/20013989/configure-node-npm-package-json-so-that-npm-test-works-on-both-unix-and-window
289. https://downloads.hindawi.com/journals/scn/2022/9473246.pdf
290. https://theproductguy.in/blogs/hash-verifier-ci/
291. https://transloadit.com/devtips/verify-file-integrity-with-go-and-sha256/
292. https://linuxsecurity.com/news/server-security/checksums-in-linux-integrity-guide
293. https://gist.github.com/joshbeckman/5c4f0244914adfd312e4?permalink_comment_id=2259606
294. https://nodejs.org/download/release/v14.19.1/docs/api/policy.html
295. https://hookdeck.com/webhooks/guides/how-to-implement-sha256-webhook-signature-verification
296. https://shattered.io/hmac-sha256-nodejs/
297. https://dev.to/osikwemhev/polices-in-nodejs-part-1-3pfj
298. https://www.aimcp.info/en/skills/372f9c66-5bc2-40a1-8b5c-183a50304b61
299. https://docs.coditect.ai/reference/skills/sha256-verification
300. https://arxiv.org/pdf/2407.02522.pdf
301. http://arxiv.org/pdf/2405.06832.pdf
302. http://arxiv.org/pdf/2402.03555.pdf
303. https://arxiv.org/pdf/1802.01790.pdf
304. https://arxiv.org/pdf/2306.13984.pdf